

SOLIDWORKS API (.NET VBA C#) — Examples (46/50): Select Assembly Components Using Advance- Set Bodies for Move Copy Example VBNET

SOLIDWORKS API Documentation (converted from CHM)

Select Assembly Components Using Advanced Selection Criteria Example (C#)

This example shows how to select assembly components that are mated to a specified part.

```
// -----
// Preconditions:
// 1. Open Public_Documents\SOLIDWORKS\SOLIDWORKS 2021\samples\tutorial\advdrawings\98food processor.sldasm.
// 2. Create InContextHasMate.xml with the following content:
//    <?xml version="1.0" encoding="UTF-8"?>
//      <SWQueryList>
//        <Query Name="InContextHasMate" Favourites_Index="1">
//          <Boolean Name="And" Category="In Context Relations" SubCategory="Has mate to part" Condition="=" Value="base plate-1@98food processor"/>
//        </Query>
//      </SWQueryList>
// 3. Place InContextHasMate.xml in c:\temp.
// 4. Open the Immediate window.
//
// Postconditions: Inspect the Immediate window and the six selections
// in the FeatureManager design tree.
// -----

using System;
using System.Diagnostics;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Forms;

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;

namespace SelectComponents_CSharp
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {

            AssemblyDoc swAssem;
            ModelDoc2 swModel;
            AdvancedSelectionCriteria criteria;
            int Count;
            string CriteriaFileName;
            bool LoadSuccess;
            int index;
            bool SelectSuccess;
            string category1;
            string category2;
            int condition;
            string val;
            bool isAnd;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssem = (AssemblyDoc)swModel;
            criteria = swAssem.GetAdvancedSelection();

            Count = criteria.GetItemCount();
```

```

        Debug.Print("Before loading a query, GetItemCount returned " + Count);

        // Query file
        CriteriaFileName = @"C:\temp\InContextHasMate.xml";

        // Load query file
        LoadSuccess = criteria.LoadCriteria(CriteriaFileName);

        Count = criteria.GetItem2Count();
        Debug.Print("After loading a query, GetItemCount returned " + Count);

        for (index = 0; index <= Count - 1; index++)
        {
            if (criteria.GetItem2(index, out category1, out category2, out condition, out val, out isAnd) > -1)
                Debug.Print( " Criterion " + index + ": " + category1 + ", " + category2 + ", " + condition + ", " + val +
                    ", " + isAnd);
            else
                Debug.Print( " Criterion " + index + " not found");
        }

        Debug.Print(((SelectionMgr)(swModel.SelectionManager)).GetSelectedObjectCount2(-1) + " objects selected before
running query");

        // Select components using selection criteria
        SelectSuccess = criteria.Select();
        Debug.Print("Select was successful: " + SelectSuccess);

        Debug.Print(((SelectionMgr)(swModel.SelectionManager)).GetSelectedObjectCount2(-1) + " objects selected after
running query");

    }

    // The SldWorks swApp variable is pre-assigned for you.
    public SldWorks swApp;

}
}

```

Select Assembly Components Using Advanced Selection Criteria Example (VB.NET)

This example shows how to select assembly components that are mated to a specified part.

```
'-----
' Preconditions:
' 1. Open Public_Documents\SOLIDWORKS\SOLIDWORKS 2021\samples\tutorial\advdrawings\98food processor.sldasm.
' 2. Create InContextHasMate.xml with the following content:
'   <?xml version="1.0" encoding="UTF-8"?>
'   <SWQueryList>
'       <Query Name="InContextHasMate" Favourites_Index="1">
'           <Boolean Name="And" Category="In Context Relations" SubCategory="Has mate to part" Condition="=" Value="base plate-
1@98food processor"/>
'       </Query>
'   </SWQueryList>
' 3. Place InContextHasMate.xml in c:\temp.
' 4. Open the Immediate window.
'
' Postconditions: Inspect the Immediate window and the six selections
' in the FeatureManager design tree.
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System.Runtime.InteropServices

Imports System

Partial Class SolidWorksMacro

Public Sub main()

Dim swAssem As AssemblyDoc

Dim criteria As AdvancedSelectionCriteria

Dim Count As Integer

Dim CriteriaFileName As String

Dim LoadSuccess As Boolean

Dim index As Integer

Dim SelectSuccess As Boolean

Dim category1 As String

Dim category2 As String

Dim condition As Integer

Dim val As String

Dim isAnd As Boolean

swAssem = swApp.ActiveDoc

criteria = swAssem.GetAdvancedSelection

Count = criteria.GetItemCount

Debug.Print("Before loading a query, GetItemCount returned " & Count)

' Query file

CriteriaFileName = "C:\temp\InContextHasMate.xml"

' Load query file

LoadSuccess = criteria.LoadCriteria(CriteriaFileName)

Count = criteria.GetItemCount

Debug.Print("After loading a query, GetItemCount returned " & Count)

For index = 0 To Count - 1

 If criteria.GetItem2(index, category1, category2, condition, val, isAnd) > -1 Then

```

        Debug.Print(vbTab & " Criterion " & index & ": " & category1 & ", " & category2 & ", " & condition & ", " & val &
", " & isAnd)
    Else
        Debug.Print(vbTab & " Criterion " & index & " not found")
    End If
Next

Debug.Print(swAssem.SelectionManager.GetSelectedObjectCount2(-1) & " objects selected before running query")

' Select components using selection criteria
SelectSuccess = criteria.Select
Debug.Print("Select was" & IIf(SelectSuccess = False, " NOT ", " ") & "successful")

Debug.Print(swAssem.SelectionManager.GetSelectedObjectCount2(-1) & " objects selected after running query")

End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class

```

Select Bodies Example (VBA)

This example shows how to select both solid and surface bodies in either a part or an assembly.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects all solid and surface bodies.
' 2. Examine the Immediate window.
'-----
Option Explicit

Sub SelectBodies(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, vBody As Variant, sPadStr As String)

    Dim swModExt As SldWorks.ModelDocExtension
    Dim swBody As SldWorks.Body2
    Dim sBodySelStr As String
    Dim sBodyTypeSelStr As String
    Dim i As Long
    Dim bRet As Boolean

    If IsEmpty(vBody) Then Exit Sub
    Set swModExt = swModel.Extension

    For i = 0 To UBound(vBody)
        Set swBody = vBody(i)
        sBodySelStr = swBody.GetSelectionId
        Debug.Print " " & sPadStr & sBodySelStr

        Select Case swBody.GetType
            Case swSolidBody
                sBodyTypeSelStr = "SOLIDBODY"
            Case swSheetBody
                sBodyTypeSelStr = "SURFACEBODY"
            Case Else
                Debug.Assert False
        End Select
        bRet = swModExt.SelectByID(sBodySelStr, sBodyTypeSelStr, 0#, 0#, 0#, True, 0, Nothing, swSelectOptionDefault): Debug.Assert bRet
    Next i
End Sub

Sub ProcessComponent(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2, nLevel As Long)

    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim sPadStr As String
    Dim vBody As Variant
    Dim i As Long

    For i = 0 To nLevel - 1
        sPadStr = sPadStr + " "
    Next i

    Debug.Print sPadStr & swComp.Name2 & " <" & swComp.ReferencedConfiguration & ">"

    ' Solid bodies
    vBody = swComp.GetBodies2(swSolidBody)
    SelectBodies swApp, swModel, vBody, sPadStr

    ' Surface bodies
    vBody = swComp.GetBodies2(swSheetBody)
    SelectBodies swApp, swModel, vBody, sPadStr

    vChildComp = swComp.GetChildren
    For i = 0 To UBound(vChildComp)
        Set swChildComp = vChildComp(i)
        ProcessComponent swApp, swModel, swChildComp, nLevel + 1
    Next i
End Sub

Sub ProcessAssembly(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2)

    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConf As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2

    Set swConfigMgr = swModel.ConfigurationManager
    Set swConf = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConf.GetRootComponent3(True)
    ProcessComponent swApp, swModel, swRootComp, 1
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swPart As SldWorks.PartDoc
    Dim vBody As Variant
    Dim i As Long
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
```

```
swModel.ClearSelection2 True

Debug.Print "File = " & swModel.GetPathName

Select Case swModel.GetType
    Case swDocPART
        Set swPart = swModel
        ' Solid bodies
        vBody = swPart.GetBodies2(swSolidBody, True)
        SelectBodies swApp, swModel, vBody, ""
        ' Surface bodies
        vBody = swPart.GetBodies2(swSheetBody, True)
        SelectBodies swApp, swModel, vBody, ""
    Case swDocASSEMBLY
        ProcessAssembly swApp, swModel
    Case Else
        Exit Sub
End Select

End Sub
```

Select Chains of Entities Attached to a Sketch Segment Example (C#)

This example shows how to select chains of entities attached to a sketch segment.

```
//-----
// Preconditions:
// 1. Open a part that contains a sketch of a rectangle.
// 2. Select one sketch segment of the rectangle.
//
// Postconditions:
// 1. Selects all of the sketch segments of the rectangle.
// 2. Examine the graphics area.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace SelectChain_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 Part;
        SelectionMgr SelMgr;
        SketchSegment sketchSegment;
        SelectData SelData;

        bool boolstatus;

        public void Main()
        {
            Part = (ModelDoc2)swApp.ActiveDoc;
            SelMgr = (SelectionMgr)Part.SelectionManager;
            SelData = (SelectData)SelMgr.CreateSelectData();
            boolstatus = Part.Extension.SelectByID2("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.01022262320328, 0.01646364019604, 0,
false, 0, null, 0);
            sketchSegment = (SketchSegment)SelMgr.GetSelectedObject6(1, -1);
            boolstatus = sketchSegment.SelectChain(true, SelData);

        }

        public SldWorks swApp;
    }
}
```

Select Chains of Entities Attached to a Sketch Segment Example (VBA)

This example shows how to select chains of entities attached to a sketch segment.

```
'-----  
' Preconditions:  
' 1. Open a part that contains a sketch of a rectangle.  
' 2. Select one sketch segment of the rectangle.  
'  
' Postconditions:  
' 1. Selects all of the sketch segments of the rectangle.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks  
Dim Part As SldWorks.ModelDoc2  
Dim SelMgr As SldWorks.SelectionMgr  
Dim sketchSegment As SldWorks.sketchSegment  
Dim SelData As SldWorks.SelectData  
Dim boolstatus As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set Part = swApp.ActiveDoc  
    Set SelMgr = Part.SelectionManager  
    Set SelData = SelMgr.CreateSelectData  
    boolstatus = Part.Extension.SelectByID2("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.01022262320328, 0.01646364019604, 0, False, 0,  
Nothing, 0)  
    Set sketchSegment = SelMgr.GetSelectedObject6(1, -1)  
    boolstatus = sketchSegment.SelectChain(True, SelData)  
  
End Sub
```

Select Chains of Entities Attached to a Sketch Segment Example (VB.NET)

This example shows how to select chains of entities attached to a sketch segment.

```
'-----
' Preconditions:
' 1. Open a part that contains a sketch of a rectangle.
' 2. Select one sketch segment of the rectangle.
'
' Postconditions:
' 1. Selects all of the sketch segments of the rectangle.
' 2. Examine the graphics area.
'-----
```

Imports SolidWorks.Interop.sldworks

Imports SolidWorks.Interop.swconst

Imports System

Partial Class SolidWorksMacro

Dim Part As ModelDoc2

Dim SelMgr As SelectionMgr

Dim sketchSegment As SketchSegment

Dim SelData As SelectData

Dim boolstatus As Boolean

Sub main()

Part = swApp.ActiveDoc

SelMgr = Part.SelectionManager

SelData = SelMgr.CreateSelectData

boolstatus = Part.Extension.SelectByID2("Line3@Sketch2", "EXTSKETCHSEGMENT", -0.01022262320328, 0.01646364019604, 0, False,

0, Nothing, 0)

sketchSegment = SelMgr.GetSelectedObject6(1, -1)

boolstatus = sketchSegment.SelectChain(True, SelData)

End Sub

Public swApp As SldWorks

End Class

Select Drawing Component Example (C#)

This example shows how to select an assembly component in a drawing.

```
//-----
// Preconditions:
// 1. Open a drawing of an assembly.
// 2. Select an assembly component in the drawing.
//
// Postconditions: Displays a message box
// containing the name of the selected assembly
// component.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Windows.Forms;

namespace GetSelectedDrawingComponent4CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModelDoc;
            SelectionMgr swSelMgr;
            Component2 swComponent;
            DrawingComponent swDrawingComponent;

            swModelDoc = (ModelDoc2)swApp.ActiveDoc;
            if (swModelDoc == null)
            {
                return;
            }
            swSelMgr = (SelectionMgr)swModelDoc.SelectionManager;

            if (swSelMgr.GetSelectedObjectCount2(0) == 0)
            {
                MessageBox.Show("No selections detected.");
                return;
            }

            if (swSelMgr.GetSelectedObjectType3(1, 0) == (int)swSelectType_e.swSelCOMPONENTS)
            {
                swDrawingComponent = (DrawingComponent)swSelMgr.GetSelectedObjectsComponent4(1, 0);

                if (swDrawingComponent == null)
                {
                    MessageBox.Show("The component is empty.");
                    return;
                }
                else
                {
                    swComponent = (Component2)swDrawingComponent.Component;
                    MessageBox.Show(swComponent.Name2);
                }
            }
            else
            {
                MessageBox.Show("The selection is not an assembly component.");
                return;
            }
        }
    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Select Drawing Component Example (VBA)

This example shows how to select an assembly component in a drawing.

```
'-----  
' Preconditions:  
' 1. Open a drawing of an assembly.  
' 2. Select an assembly component in the drawing.  
'  
' Postconditions: Displays a message box  
' containing the name of the selected assembly  
' component.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModelDoc As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swComponent As SldWorks.Component2  
Dim swDrawingComponent As SldWorks.DrawingComponent  
  
Sub main()  
  
    Set swApp = Application.SldWorks  
  
    Set swModelDoc = swApp.ActiveDoc  
    If swModelDoc Is Nothing Then  
        Exit Sub  
    End If  
    Set swSelMgr = swModelDoc.SelectionManager  
  
    If swSelMgr.GetSelectedObjectCount2(0) = 0 Then  
        MsgBox "No selections detected."  
        Exit Sub  
    End If  
  
    If swSelMgr.GetSelectedObjectType3(1, 0) = swSelectType_e.swSelCOMPONENTS Then  
        Set swDrawingComponent = swSelMgr.GetSelectedObjectsComponent4(1, 0)  
        If swDrawingComponent Is Nothing Then  
            MsgBox "The component is empty."  
            Exit Sub  
        Else  
            Set swComponent = swDrawingComponent.Component  
            MsgBox swComponent.Name2  
        End If  
    Else  
        MsgBox "The selection is not an assembly component."  
        Exit Sub  
    End If  
  
End Sub
```

Select Drawing Component Example (VB.NET)

This example shows how to select an assembly component in a drawing.

```
-----
Preconditions:
1. Open a drawing of an assembly.
2. Select an assembly component in the drawing.
Postconditions: Displays a message box
containing the name of the selected assembly
component.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Dim swModelDoc As ModelDoc2
    Dim swSelMgr As SelectionMgr
    Dim swComponent As Component2
    Dim swDrawingComponent As DrawingComponent

    Sub main()

        swModelDoc = swApp.ActiveDoc
        If swModelDoc Is Nothing Then
            Exit Sub
        End If
        swSelMgr = swModelDoc.SelectionManager

        If swSelMgr.GetSelectedObjectCount2(0) = 0 Then
            MsgBox("No selections detected.")
            Exit Sub
        End If

        If swSelMgr.GetSelectedObjectType3(1, 0) = swSelectType_e.swSelCOMPONENTS Then
            swDrawingComponent = swSelMgr.GetSelectedObjectsComponent4(1, 0) 'works
            If swDrawingComponent Is Nothing Then
                MsgBox("The component is empty.")
                Exit Sub
            Else
                swComponent = swDrawingComponent.Component
                MsgBox(swComponent.Name2)
            End If
        Else
            MsgBox("The selection is not an assembly component.")
            Exit Sub
        End If

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Select Edges of All Holes on Face Example (C#)

This example shows how to select the edges on all circular holes on a face.

NOTE: A circular hole can be defined in a many ways; for example, a circle inside an extruded sketch or a circle that is part of a cut-extrude feature. Thus, there is no ready way to detect a hole solely based on the feature information. However, by directly examining the geometry and topology of the model, it is possible to deduce holes. This code uses many of the geometry- and topology-related objects and methods to traverse a face and look for circular holes.

```
//-----
// Preconditions: Verify that the specified part to open exists.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Changes the view orientation to Back.
// 3. Selects a face.
// 4. Examines the geometry and topology of the selected face and
//    identifies the holes in the face.
// 5. Deselects the face.
// 6. Selects all edges of all circular holes on the face.
// 7. Examine the graphics area.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace CoEdgeCSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SelectionMgr swSelMgr = default(SelectionMgr);
            SelectData swSelData = default(SelectData);
            Face2 swFace = default(Face2);
            string fileName = null;
            int errors = 0;
            int warnings = 0;
            bool bRet = false;

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\gear-
caddy.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Si
            swModel.ShowNamedView2("**Back", 2);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            bRet = swModelDocExt.SelectByID2("", "FACE", 0.0290197084065686, 0.0111645373580202, 0, false, 0, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelMgr.GetSelectedObject6(1, -1);
            swSelData = (SelectData)swSelMgr.CreateSelectData();
            swModel.ClearSelection2(true);
            SelectHoleEdges(swFace, swSelData);
        }

        public double[] GetFaceNormalAtMidCoEdge(CoEdge swCoEdge)
        {
            Face2 swFace = default(Face2);
            Surface swSurface = default(Surface);
            Loop2 swLoop = default(Loop2);
            double[] varParams = null;
            double[] varPoint = null;
            double dblMidParam = 0;
            double[] dblNormal = new double[3];
            bool bFaceSenseReversed = false;

            varParams = (double[])swCoEdge.GetCurveParams();
            if (varParams[6] > varParams[7])
            {
                dblMidParam = (varParams[6] - varParams[7]) / 2 + varParams[7];
            }
            else
            {
                dblMidParam = (varParams[7] - varParams[6]) / 2 + varParams[6];
            }
            varPoint = (double[])swCoEdge.Evaluate(dblMidParam);

            // Get the face of the given coedge
            // Check for the sense of the face
            swLoop = (Loop2)swCoEdge.GetLoop();
            swFace = (Face2)swLoop.GetFace();
            swSurface = (Surface)swFace.FaceInSurfaceSense();
            bFaceSenseReversed = swFace.FaceInSurfaceSense();
            varParams = (double[])swSurface.EvaluateAtPoint(varPoint[0], varPoint[1], varPoint[2]);
            if (bFaceSenseReversed)
            {
                // Negate the surface normal as it is opposite from the face normal
                dblNormal[0] = -varParams[0];
            }
        }
    }
}
```

```

        dblNormal[1] = -varParams[1];
        dblNormal[2] = -varParams[2];
    }
    else
    {
        dblNormal[0] = varParams[0];
        dblNormal[1] = varParams[1];
        dblNormal[2] = varParams[2];
    }
    return dblNormal;
}

public double[] GetTangentAtMidCoEdge(CoEdge swCoEdge)
{
    double[] varParams = null;
    double dblMidParam = 0;
    double[] dblTangent = new double[3];
    varParams = (double[])swCoEdge.GetCurveParams();
    if (varParams[6] > varParams[7])
    {
        dblMidParam = (varParams[6] - varParams[7]) / 2.0 + varParams[7];
    }
    else
    {
        dblMidParam = (varParams[7] - varParams[6]) / 2.0 + varParams[6];
    }
    varParams = (double[])swCoEdge.Evaluate(dblMidParam);
    dblTangent[0] = varParams[3];
    dblTangent[1] = varParams[4];
    dblTangent[2] = varParams[5];
    return dblTangent;
}

public double[] GetCrossProduct(double[] varVec1, double[] varVec2)
{
    double[] dblCross = new double[3];

    dblCross[0] = varVec1[1] * varVec2[2] - varVec1[2] * varVec2[1];
    dblCross[1] = varVec1[2] * varVec2[0] - varVec1[0] * varVec2[2];
    dblCross[2] = varVec1[0] * varVec2[1] - varVec1[1] * varVec2[0];
    return dblCross;
}

public bool VectorsAreEqual(double[] varVec1, double[] varVec2)
{
    bool functionReturnValue = false;
    double dblMag = 0;
    double dblDot = 0;
    double[] dblUnit1 = new double[3];
    double[] dblUnit2 = new double[3];
    dblMag = Math.Pow((varVec1[0] * varVec1[0] + varVec1[1] * varVec1[1] + varVec1[2] * varVec1[2]), 0.5);
    dblUnit1[0] = varVec1[0] / dblMag;
    dblUnit1[1] = varVec1[1] / dblMag;
    dblUnit1[2] = varVec1[2] / dblMag;
    dblMag = Math.Pow((varVec2[0] * varVec2[0] + varVec2[1] * varVec2[1] + varVec2[2] * varVec2[2]), 0.5);
    dblUnit2[0] = varVec2[0] / dblMag;
    dblUnit2[1] = varVec2[1] / dblMag;
    dblUnit2[2] = varVec2[2] / dblMag;
    dblDot = dblUnit1[0] * dblUnit2[0] + dblUnit1[1] * dblUnit2[1] + dblUnit1[2] * dblUnit2[2];
    dblDot = Math.Abs(dblDot - 1.0);
    // Compare within a tolerance
    //1.0e-10
    if (dblDot < 1E-10)
    {
        functionReturnValue = true;
    }
    else
    {
        functionReturnValue = false;
    }
    return functionReturnValue;
}

public void SelectHoleEdges(Face2 swFace, SelectData swSelData)
{
    Loop2 swThisLoop = default(Loop2);
    CoEdge swThisCoEdge = default(CoEdge);
    CoEdge swPartnerCoEdge = default(CoEdge);
    Entity swEntity = default(Entity);
    double[] varThisNormal = null;
    double[] varPartnerNormal = null;
    double[] varCrossProduct = null;
    double[] varTangent = null;
    object[] vEdgeArr = null;
    Edge swEdge = default(Edge);
    Curve swCurve = default(Curve);
    bool bRet = true;
    int count = 0;
    bool bCount = true;

    swThisLoop = (Loop2)swFace.GetFirstLoop();
    while ((swThisLoop != null))

```

```

{
    // Hole is inner loop
    // Circular or elliptical hole has only one edge
    bRet = swThisLoop.IsOuter();
    count = swThisLoop.GetEdgeCount();
    if (count != 1)
    {
        bCount = false;
    }
    else
    {
        bCount = true;
    }
    if ((bRet == false) && (bCount == true))
    {
        swThisCoEdge = (CoEdge)swThisLoop.GetFirstCoEdge();
        swPartnerCoEdge = (CoEdge)swThisCoEdge.GetPartner();
        varThisNormal = (double[])GetFaceNormalAtMidCoEdge(swThisCoEdge);
        varPartnerNormal = (double[])GetFaceNormalAtMidCoEdge(swPartnerCoEdge);
        if (!VectorsAreEqual(varThisNormal, varPartnerNormal))
        {
            // There is a sufficient change between the two faces to determine
            // what kind of transition is being made
            varCrossProduct = (double[])GetCrossProduct(varThisNormal, varPartnerNormal);
            varTangent = (double[])GetTangentAtMidCoEdge(swThisCoEdge);
            if (VectorsAreEqual(varCrossProduct, varTangent))
            {
                // Hole
                vEdgeArr = (object[])swThisLoop.GetEdges();
                Debug.Assert(0 < vEdgeArr.Length);
                swEdge = (Edge)vEdgeArr[0];
                swCurve = (Curve)swEdge.GetCurve();
                // Ignore elliptical holes
                if (swCurve.IsCircle())
                {
                    swEntity = (Entity)swEdge;
                    bRet = swEntity.Select4(true, swSelData);
                    Debug.Assert(bRet);
                }
            }
        }
    }

    // Move on to the next
    swThisLoop = (Loop2)swThisLoop.GetNext();
}

}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Select Edges of All Holes on Face Example_VB (Topic)

SOLIDWORKS API Help

Select Edges of All Holes on Face Example (VBA)

This example shows how to select the edges on all circular holes on a face.

NOTE: A circular hole can be defined in a many ways; for example, a circle inside an extruded sketch or a circle that is part of a cut-extrude feature. Thus, there is no ready way to detect a hole solely based on the feature information. However, by directly examining the geometry and topology of the model, it is possible to deduce holes. This code uses many of the geometry- and topology-related objects and methods to traverse a face and look for circular holes.

```
'-----
' Preconditions: Verify that the specified part to open exists.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Changes the view orientation to Back.
' 3. Selects a face.
' 4. Examines the geometry and topology of the selected face and
'    identifies the holes in the face.
' 5. Deselects the face.
' 6. Selects all edges of all circular holes on the face.
' 7. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Function GetFaceNormalAtMidCoEdge(swCoEdge As SldWorks.CoEdge) As Variant
    Dim swFace As SldWorks.Face2
    Dim swSurface As SldWorks.Surface
    Dim swLoop As SldWorks.Loop2
    Dim varParams As Variant
    Dim varPoint As Variant
    Dim dblMidParam As Double
    Dim dblNormal(2) As Double
    Dim bFaceSenseReversed As Boolean

    varParams = swCoEdge.GetCurveParams
    If varParams(6) > varParams(7) Then
        dblMidParam = (varParams(6) - varParams(7)) / 2 + varParams(7)
    Else
        dblMidParam = (varParams(7) - varParams(6)) / 2 + varParams(6)
    End If
    varPoint = swCoEdge.Evaluate(dblMidParam)

    ' Get the face of the given coedge
    ' Check for the sense of the face
    Set swLoop = swCoEdge.GetLoop
    Set swFace = swLoop.GetFace
    Set swSurface = swFace.GetSurface
    bFaceSenseReversed = swFace.FaceInSurfaceSense
    varParams = swSurface.EvaluateAtPoint(varPoint(0), varPoint(1), varPoint(2))

    If bFaceSenseReversed Then
        ' Negate the surface normal as it is opposite from the face normal
        dblNormal(0) = -varParams(0)
        dblNormal(1) = -varParams(1)
        dblNormal(2) = -varParams(2)
    Else
        dblNormal(0) = varParams(0)
        dblNormal(1) = varParams(1)
        dblNormal(2) = varParams(2)
    End If

    GetFaceNormalAtMidCoEdge = dblNormal
End Function

Function GetTangentAtMidCoEdge(swCoEdge As SldWorks.CoEdge) As Variant
    Dim varParams As Variant
    Dim dblMidParam As Double
    Dim dblTangent(2) As Double

    varParams = swCoEdge.GetCurveParams
    If varParams(6) > varParams(7) Then
        dblMidParam = (varParams(6) - varParams(7)) / 2# + varParams(7)
    Else
        dblMidParam = (varParams(7) - varParams(6)) / 2# + varParams(6)
    End If

    varParams = swCoEdge.Evaluate(dblMidParam)
    dblTangent(0) = varParams(3)
    dblTangent(1) = varParams(4)
    dblTangent(2) = varParams(5)
    GetTangentAtMidCoEdge = dblTangent
End Function

Function GetCrossProduct(varVec1 As Variant, varVec2 As Variant) As Variant
    Dim dblCross(2) As Double

    dblCross(0) = varVec1(1) * varVec2(2) - varVec1(2) * varVec2(1)
    dblCross(1) = varVec1(2) * varVec2(0) - varVec1(0) * varVec2(2)
    dblCross(2) = varVec1(0) * varVec2(1) - varVec1(1) * varVec2(0)
    GetCrossProduct = dblCross
End Function

Function VectorsAreEqual(varVec1 As Variant, varVec2 As Variant) As Boolean
    Dim dblMag As Double
    Dim dblDot As Double
```

```

Dim dblUnit1(2) As Double
Dim dblUnit2(2) As Double

dblMag = (varVec1(0) * varVec1(0) + varVec1(1) * varVec1(1) + varVec1(2) * varVec1(2)) ^ 0.5
dblUnit1(0) = varVec1(0) / dblMag: dblUnit1(1) = varVec1(1) / dblMag: dblUnit1(2) = varVec1(2) / dblMag
dblMag = (varVec2(0) * varVec2(0) + varVec2(1) * varVec2(1) + varVec2(2) * varVec2(2)) ^ 0.5
dblUnit2(0) = varVec2(0) / dblMag: dblUnit2(1) = varVec2(1) / dblMag: dblUnit2(2) = varVec2(2) / dblMag
dblDot = dblUnit1(0) * dblUnit2(0) + dblUnit1(1) * dblUnit2(1) + dblUnit1(2) * dblUnit2(2)
dblDot = Abs(dblDot - 1#)

' Compare within a tolerance
If dblDot < 0.000000001 Then '1.0e-10
    VectorsAreEqual = True
Else
    VectorsAreEqual = False
End If

End Function

Sub SelectHoleEdges(swFace As SldWorks.Face2, swSelData As SldWorks.SelectData)
Dim swThisLoop As SldWorks.Loop2
Dim swThisCoEdge As SldWorks.CoEdge
Dim swPartnerCoEdge As SldWorks.CoEdge
Dim swEntity As SldWorks.Entity
Dim varThisNormal As Variant
Dim varPartnerNormal As Variant
Dim varCrossProduct As Variant
Dim varTangent As Variant
Dim vEdgeArr As Variant
Dim swEdge As SldWorks.Edge
Dim swCurve As SldWorks.Curve
Dim bRet As Boolean

Set swThisLoop = swFace.GetFirstLoop
Do While Not swThisLoop Is Nothing
    ' Hole is inner loop
    ' Circular or elliptical hole has only one edge
    If swThisLoop.IsOuter = False And 1 = swThisLoop.GetEdgeCount Then
        Set swThisCoEdge = swThisLoop.GetFirstCoEdge
        Set swPartnerCoEdge = swThisCoEdge.GetPartner
        varThisNormal = GetFaceNormalAtMidCoEdge(swThisCoEdge)
        varPartnerNormal = GetFaceNormalAtMidCoEdge(swPartnerCoEdge)
        If Not VectorsAreEqual(varThisNormal, varPartnerNormal) Then
            ' There is a sufficient change between the two faces to determine
            ' what kind of transition is being made
            varCrossProduct = GetCrossProduct(varThisNormal, varPartnerNormal)
            varTangent = GetTangentAtMidCoEdge(swThisCoEdge)
            If VectorsAreEqual(varCrossProduct, varTangent) Then
                ' Hole
                vEdgeArr = swThisLoop.GetEdges
                Debug.Assert 0 = UBound(vEdgeArr)
                Set swEdge = vEdgeArr(0)
                Set swCurve = swEdge.GetCurve
                ' Ignore elliptical holes
                If swCurve.IsCircle Then
                    Set swEntity = swEdge
                    bRet = swEntity.Select4(True, swSelData)
                    Debug.Assert bRet
                End If
            End If
        End If
    End If
    ' Move on to the next
    Set swThisLoop = swThisLoop.GetNext
Loop

End Sub

Sub main()
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swSelData As SldWorks.SelectData
Dim swFace As SldWorks.Face2
Dim fileName As String
Dim errors As Long
Dim warnings As Long
Dim bRet As Boolean

Set swApp = CreateObject("SldWorks.Application")
fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\gear- caddy.sldprt"
Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
swModel.ShowNamedView2 "Back", 2
Set swModelDocExt = swModel.Extension
bRet = swModelDocExt.SelectByID2("", "FACE", 2.90197084065686E-02, 1.11645373580202E-02, 0, False, 0, Nothing, 0)
Set swSelMgr = swModel.SelectionManager
Set swFace = swSelMgr.GetSelectedObject6(1, -1)
Set swSelData = swSelMgr.CreateSelectData

swModel.ClearSelection2 True

SelectHoleEdges swFace, swSelData

End Sub

```

Select Edges of All Holes on Face Example_VBNET (Topic)

SOLIDWORKS API Help

Select Edges of All Holes on Face Example (VB.NET)

This example shows how to select the edges on all circular holes on a face.

NOTE: A circular hole can be defined in a many ways; for example, a circle inside an extruded sketch or a circle that is part of a cut-extrude feature. Thus, there is no ready way to detect a hole solely based on the feature information. However, by directly examining the geometry and topology of the model, it is possible to deduce holes. This code uses many of the geometry- and topology-related objects and methods to traverse a face and look for circular holes.

```
'-----
' Preconditions: Verify that the specified part to open exists.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Changes the view orientation to Back.
' 3. Selects a face.
' 4. Examines the geometry and topology of the selected face and
'    identifies the holes in the face.
' 5. Deselects the face.
' 6. Selects all edges of all circular holes on the face.
' 7. Examine the graphics area.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSelMgr As SelectionMgr
        Dim swSelData As SelectData
        Dim swFace As Face2
        Dim fileName As String
        Dim errors As Integer
        Dim warnings As Integer
        Dim bRet As Boolean

        swApp = CreateObject("SldWorks.Application")
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\gear- caddy.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModel.ShowNamedView2("Back", 2)
        swModelDocExt = swModel.Extension
        bRet = swModelDocExt.SelectByID("", "FACE", 0.0290197084065686, 0.0111645373580202, 0, False, 0, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        swFace = swSelMgr.GetSelectedObject6(1, -1)
        swSelData = swSelMgr.CreateSelectData
        swModel.ClearSelection2(True)
        SelectHoleEdges(swFace, swSelData)

    End Sub

    Function GetFaceNormalAtMidCoEdge(ByVal swCoEdge As CoEdge) As Double()
        Dim swFace As Face2
        Dim swSurface As Surface
        Dim swLoop As Loop2
        Dim varParams() As Double
        Dim varPoint() As Double
        Dim dblMidParam As Double
        Dim dblNormal(2) As Double
        Dim bFaceSenseReversed As Boolean

        varParams = swCoEdge.GetCurveParams
        If varParams(6) > varParams(7) Then
            dblMidParam = (varParams(6) - varParams(7)) / 2 + varParams(7)
        Else
            dblMidParam = (varParams(7) - varParams(6)) / 2 + varParams(6)
        End If
        varPoint = swCoEdge.Evaluate(dblMidParam)

        ' Get the face of the given coedge
        ' Check for the sense of the face
        swLoop = swCoEdge.GetLoop
        swSurface = swLoop.GetFace
        bFaceSenseReversed = swFace.FaceInSurfaceSense
        varParams = swSurface.EvaluateAtPoint(varPoint(0), varPoint(1), varPoint(2))
        If bFaceSenseReversed Then
            ' Negate the surface normal as it is opposite from the face normal
            dblNormal(0) = -varParams(0)
            dblNormal(1) = -varParams(1)
            dblNormal(2) = -varParams(2)
        Else
            dblNormal(0) = varParams(0)
            dblNormal(1) = varParams(1)
            dblNormal(2) = varParams(2)
        End If
        GetFaceNormalAtMidCoEdge = dblNormal
    End Function

    Function GetTangentAtMidCoEdge(ByVal swCoEdge As CoEdge) As Double()
        Dim varParams() As Double
        Dim dblMidParam As Double
        Dim dblTangent(2) As Double

        varParams = swCoEdge.GetCurveParams
        If varParams(6) > varParams(7) Then
            dblMidParam = (varParams(6) - varParams(7)) / 2.0# + varParams(7)
        Else
            dblMidParam = (varParams(7) - varParams(6)) / 2.0# + varParams(6)
        End If
    End Function
```

```

End If
varParams = swCoEdge.Evaluate(dblMidParam)
dblTangent(0) = varParams(3)
dblTangent(1) = varParams(4)
dblTangent(2) = varParams(5)
GetTangentAtMidCoEdge = dblTangent

End Function

Function GetCrossProduct(ByVal varVec1() As Double, ByVal varVec2() As Double) As Double()
Dim dblCross(2) As Double

dblCross(0) = varVec1(1) * varVec2(2) - varVec1(2) * varVec2(1)
dblCross(1) = varVec1(2) * varVec2(0) - varVec1(0) * varVec2(2)
dblCross(2) = varVec1(0) * varVec2(1) - varVec1(1) * varVec2(0)
GetCrossProduct = dblCross

End Function

Function VectorsAreEqual(ByVal varVec1() As Double, ByVal varVec2() As Double) As Boolean
Dim dblMag As Double
Dim dblDot As Double
Dim dblUnit1(2) As Double
Dim dblUnit2(2) As Double
dblMag = (varVec1(0) * varVec1(0) + varVec1(1) * varVec1(1) + varVec1(2) * varVec1(2)) ^ 0.5
dblUnit1(0) = varVec1(0) / dblMag : dblUnit1(1) = varVec1(1) / dblMag : dblUnit1(2) = varVec1(2) / dblMag
dblMag = (varVec2(0) * varVec2(0) + varVec2(1) * varVec2(1) + varVec2(2) * varVec2(2)) ^ 0.5
dblUnit2(0) = varVec2(0) / dblMag : dblUnit2(1) = varVec2(1) / dblMag : dblUnit2(2) = varVec2(2) / dblMag
dblDot = dblUnit1(0) * dblUnit2(0) + dblUnit1(1) * dblUnit2(1) + dblUnit1(2) * dblUnit2(2)
dblDot = Math.Abs(dblDot - 1.0#)
' Compare within a tolerance
If dblDot < 0.0000000001 Then '1.0e-10
    VectorsAreEqual = True
Else
    VectorsAreEqual = False
End If

End Function

Sub SelectHoleEdges(ByVal swFace As Face2, ByVal swSelData As SelectData)
Dim swThisLoop As Loop2
Dim swThisCoEdge As CoEdge
Dim swPartnerCoEdge As CoEdge
Dim swEntity As Entity
Dim varThisNormal() As Double
Dim varPartnerNormal() As Double
Dim varCrossProduct() As Double
Dim varTangent() As Double
Dim vEdgeArr() As Object
Dim swEdge As Edge
Dim swCurve As Curve
Dim bRet As Boolean

swThisLoop = swFace.GetFirstLoop
Do While Not swThisLoop Is Nothing
    ' Hole is inner loop
    ' Circular or elliptical hole has only one edge
    If swThisLoop.IsOuter = False And 1 = swThisLoop.GetEdgeCount Then
        swThisCoEdge = swThisLoop.GetFirstCoEdge
        swPartnerCoEdge = swThisCoEdge.GetPartner
        varThisNormal = GetFaceNormalAtMidCoEdge(swThisCoEdge)
        varPartnerNormal = GetFaceNormalAtMidCoEdge(swPartnerCoEdge)
        If Not VectorsAreEqual(varThisNormal, varPartnerNormal) Then
            ' There is a sufficient change between the two faces to determine
            ' what kind of transition is being made
            varCrossProduct = GetCrossProduct(varThisNormal, varPartnerNormal)
            varTangent = GetTangentAtMidCoEdge(swThisCoEdge)
            If VectorsAreEqual(varCrossProduct, varTangent) Then
                ' Hole
                vEdgeArr = swThisLoop.GetEdges
                Debug.Assert(0 = UBound(vEdgeArr))
                swEdge = vEdgeArr(0)
                swCurve = swEdge.GetCurve
                ' Ignore elliptical holes
                If swCurve.IsCircle Then
                    swEntity = swEdge
                    bRet = swEntity.Select4(True, swSelData)
                    Debug.Assert(bRet)
                End If
            End If
        End If
    End If
    ' Move on to the next
    swThisLoop = swThisLoop.GetNext
Loop
End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks

```

End Class

Select Entity in Drawing View Example (C#)

This example shows how to select a model face, edge, or vertex in a drawing view and dimension it.

```
//-----
// Preconditions:
// 1. Open a part or fully resolved assembly.
// 2. Verify that the specified template exists.
// 3. Select a face, edge, or vertex.
//
// Postconditions:
// 1. Creates a new drawing with three views.
// 2. Dimensions the selected face, edge, or vertex
//    in the first drawing view.
// 3. Examine the drawing.
//
// NOTE: The dimension is not guaranteed to be created if a face is selected.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
namespace AddDimension_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public void Main()
        {
            const string sPathToTemplate = "C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2016\\data\\templates\\drawing.drwdot";
            const double nYoffset = 0.01;

            ModelDoc2 swModel = default(ModelDoc2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Entity swEnt = default(Entity);

            DrawingDoc swDraw = default(DrawingDoc);
            ModelDoc2 swDrawModel = default(ModelDoc2);
            View swView = default(View);
            double[] vOutline = null;
            DisplayDimension swDispDim = default(DisplayDimension);

            double nXpos = 0;
            double nYpos = 0;

            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            swEnt = (Entity)swSelMgr.GetSelectedObject6(1, -1);

            if ((swEnt != null))
            {
                swDraw = (DrawingDoc)swApp.NewDrawing2((int)swDwgTemplates_e.swDwgTemplateCustom, sPathToTemplate, (int)swDwgPaperSi
                swDrawModel = (ModelDoc2)swDraw;

                bRet = swDraw.Create3rdAngleViews2(swModel.GetPathName());

                swView = (View)swDraw.GetFirstView();
                swView = (View)swView.GetNextView();

                bRet = swView.SelectEntity(swEnt, false);

                // Work out where to place dimension -
                // midway across view and slightly above
                vOutline = (double[])swView.GetOutline();
                nXpos = (vOutline[0] + vOutline[2]) / 2.0;
                nYpos = vOutline[3] + nYoffset;

                // Create the dimension, even if the entity is not
                // visible in the drawing view
                swDispDim = (DisplayDimension)swDrawModel.Extension.AddDimension(nXpos, nYpos, 0.0, (int)swSmartDimensionDirection_e

            }
        }
    }

    public SldWorks swApp;
}
}
```

Select Entity in Drawing View Example (VBA)

This example shows how to select a model face, edge, or vertex in a drawing view and dimension it.

```
'-----
' Preconditions:
' 1. Open a part or fully resolved assembly.
' 2. Verify that the specified template exists.
' 3. Select a face, edge, or vertex.
'
' Postconditions:
' 1. Creates a new drawing with three views.
' 2. Dimensions the selected face, edge, or vertex
'    in the first drawing view.
' 3. Examine the drawing.
'
' NOTE: The dimension is not guaranteed to be created if a face is selected.
'-----
Option Explicit

Sub main()
    Const sPathToTemplate As String = "C:\ProgramData\SolidWorks\SOLIDWORKS 2016\data\templates\drawing.drwdot"
    Const nYoffset As Double = 0.01

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEnt As SldWorks.Entity

    Dim swDraw As SldWorks.DrawingDoc
    Dim swDrawModel As SldWorks.ModelDoc2
    Dim swView As SldWorks.View
    Dim vOutline As Variant
    Dim swDispDim As SldWorks.DisplayDimension

    Dim nXpos As Double
    Dim nYpos As Double

    Dim bRet As Boolean

    Set swApp = CreateObject("SldWorks.Application")
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swEnt = swSelMgr.GetSelectedObject6(1, -1)

    If Not swEnt Is Nothing Then
        Set swDraw = swApp.NewDrawing2(swDwgTemplateCustom, sPathToTemplate, swDwgPaperA1size, 0#, 0#)
        Set swDrawModel = swDraw

        bRet = swDraw.Create3rdAngleViews2(swModel.GetPathName)

        Set swView = swDraw.GetFirstView
        Set swView = swView.GetNextView

        bRet = swView.SelectEntity(swEnt, False)

        ' Work out where to place dimension -
        ' midway across view and slightly above
        vOutline = swView.GetOutline
        nXpos = (vOutline(0) + vOutline(2)) / 2#
        nYpos = vOutline(3) + nYoffset

        ' Create the dimension, even if the entity is not
        ' visible in the drawing view
        Set swDispDim = swDrawModel.Extension.AddDimension(nXpos, nYpos, 0#, swSmartDimensionDirection_Left)

    End If
End Sub
```

Select Entity in Drawing View Example (VB.NET)

This example shows how to select a model face, edge, or vertex in a drawing view and dimension it.

```
'-----
' Preconditions:
' 1. Open a part or fully resolved assembly.
' 2. Verify that the specified template exists.
' 3. Select a face, edge, or vertex.
'
' Postconditions:
' 1. Creates a new drawing with three views.
' 2. Dimensions the selected face, edge, or vertex
'    in the first drawing view.
' 3. Examine the drawing.
'
' NOTE: The dimension is not guaranteed to be created if a face is selected.
'-----
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        Const sPathToTemplate As String = "C:\ProgramData\SolidWorks\SOLIDWORKS 2016\data\templates\drawing.drwdot"
        Const nYoffset As Double = 0.01
```

```
        Dim swModel As ModelDoc2
        Dim swSelMgr As SelectionMgr
        Dim swEnt As Entity
```

```
        Dim swDraw As DrawingDoc
        Dim swDrawModel As ModelDoc2
        Dim swView As View
        Dim vOutline As Object
        Dim swDispDim As DisplayDimension
```

```
        Dim nXpos As Double
        Dim nYpos As Double
```

```
        Dim bRet As Boolean
```

```
        swModel = swApp.ActiveDoc
        swSelMgr = swModel.SelectionManager
        swEnt = swSelMgr.GetSelectedObject6(1, -1)
```

```
        If Not swEnt Is Nothing Then
```

```
            swDraw = swApp.NewDrawing2(swDwgTemplates_e.swDwgTemplateCustom, sPathToTemplate, swDwgPaperSizes_e.swDwgPaperA1size, 0.
            swDrawModel = swDraw
```

```
            bRet = swDraw.Create3rdAngleViews2(swModel.GetPathName)
```

```
            swView = swDraw.GetFirstView
            swView = swView.GetNextView
```

```
            bRet = swView.SelectEntity(swEnt, False)
```

```
            ' Work out where to place dimension -
            ' midway across view and slightly above
            vOutline = swView.GetOutline
            nXpos = (vOutline(0) + vOutline(2)) / 2.0#
            nYpos = vOutline(3) + nYoffset
```

```
            ' Create the dimension, even if the entity is not
            ' visible in the drawing view
```

```
            swDispDim = swDrawModel.Extension.AddDimension(nXpos, nYpos, 0.0#, swSmartDimensionDirection_e.swSmartDimensionDirector
```

```
        End If
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Select Face Using Intersecting Ray Example (C#)

This example shows how to select the first face that is intersected by a ray that starts at the specified point and runs parallel to the specified direction vector within the specified radius.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\api\box.sldrpt.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects the first face intersected by a ray that starts
//    at the specified point and runs parallel to the specified
//    direction vector within the specified radius.
// 2. Examine the graphics area and Immediate window.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            double startPointX = 0;
            double startPointY = 0;
            double startPointZ = 0;
            double rayVecX = 0;
            double rayVecY = 0;
            double rayVecZ = 0;
            double radius = 0;
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            startPointX = 0.1;
            startPointY = 0.0;
            startPointZ = 0.0;

            rayVecX = -1.0;
            rayVecY = 0.0;
            rayVecZ = 0.0;

            radius = 1.0;

            status = swModelDocExt.SelectByRay(startPointX, startPointY, startPointZ, rayVecX, rayVecY, rayVecZ, radius, (int)swSelectType_e.swSelFACES, false, 0, (int)swSelectOption_e.swSelectOptionDefault);
            Debug.Print("Selection status: " + status);
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Select Face Using Intersecting Ray Example (VBA)

This example shows how to select the first face that is intersected by a ray that starts at the specified point and runs parallel to the specified direction vector within the specified radius.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\api\box.sldrpt.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects the first face intersected by a ray that starts  
'    at the specified point and runs parallel to the specified  
'    direction vector within the specified radius.  
' 2. Examine the graphics area and Immediate window.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swModelDocExt As SldWorks.ModelDocExtension  
    Dim startPointX As Double  
    Dim startPointY As Double  
    Dim startPointZ As Double  
    Dim rayVecX As Double  
    Dim rayVecY As Double  
    Dim rayVecZ As Double  
    Dim radius As Double  
    Dim status As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExt = swModel.Extension  
  
    startPointX = 0.1  
    startPointY = 0#  
    startPointZ = 0#  
  
    rayVecX = -1#  
    rayVecY = 0#  
    rayVecZ = 0#  
  
    radius = 1#  
  
    status = swModelDocExt.SelectByRay(startPointX, startPointY, startPointZ, rayVecX, rayVecY, rayVecZ, radius, swSelectType_e.swSelFACES, False, 0,  
swSelectOption_e.swSelectOptionDefault)  
    Debug.Print "Selection status: " & status  
  
End Sub
```

Select_Face_Using_Intersecting_Ray_Example_VBNET (Topic)

SOLIDWORKS API Help

Select Face Using Intersecting Ray Example (VB.NET)

This example shows how to select the first face that is intersected by a ray that starts at the specified point and runs parallel to the specified direction vector within the specified radius.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\api\box.sldrpt.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Selects the first face intersected by a ray that starts
'    at the specified point and runs parallel to the specified
'    direction vector within the specified radius.
' 2. Examine the graphics area and Immediate window.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim startPointX As Double
        Dim startPointY As Double
        Dim startPointZ As Double
        Dim rayVecX As Double
        Dim rayVecY As Double
        Dim rayVecZ As Double
        Dim radius As Double
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension

        startPointX = 0.1
        startPointY = 0.0#
        startPointZ = 0.0#

        rayVecX = -1.0#
        rayVecY = 0.0#
        rayVecZ = 0.0#

        radius = 1.0#

        status = swModelDocExt.SelectByRay(startPointX, startPointY, startPointZ, rayVecX, rayVecY, rayVecZ, radius, swSelectType_e.swSelFACES, False, 0,
        swSelectOption_e.swSelectOptionDefault)
        Debug.Print("Selection status: " & status)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Select Loop of Edges Example (VBA)

This example shows how to use various geometry- and topology-related methods to select a set of edges that form a closed loop around a face.

```
'-----
' Preconditions:
' 1. Open a part or assembly.
' 2. Hold down the Ctrl key and select an edge and a
'    face adjacent to that edge.
'
' Postconditions:
' 1. Selects a loop of edges on the face.
' 2. Examine the graphics area.
'-----
Option Explicit

Sub SelectLoop(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swLoop As SldWorks.Loop2, swSelData As SldWorks.SelectData)
    Dim vEdgeArr As Variant
    Dim vEdge As Variant
    Dim swEdge As SldWorks.Edge
    Dim swEnt As SldWorks.Entity
    Dim bRet As Boolean

    vEdgeArr = swLoop.GetEdges
    Debug.Assert Not IsEmpty(vEdgeArr)
    For Each vEdge In vEdgeArr
        Set swEdge = vEdge
        Set swEnt = swEdge
        bRet = swEnt.Select4(True, swSelData)
    Next
End Sub

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swEdge As SldWorks.Edge
    Dim swFace As SldWorks.Face2
    Dim swSelData As SldWorks.SelectData
    Dim vCoEdgeArr As Variant
    Dim vCoEdge As Variant
    Dim swCoEdge As SldWorks.CoEdge
    Dim swLoop As SldWorks.Loop2

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelMgr = swModel.SelectionManager
    Set swEdge = swSelMgr.GetSelectedObject6(1, -1)
    Set swSelData = swSelMgr.CreateSelectData
    Set swFace = swSelMgr.GetSelectedObject6(2, -1)
    swModel.ClearSelection2 True

    vCoEdgeArr = swEdge.GetCoEdges

    ' 1 or 2: Coedges for an edge on a surface body
    ' 2: Coedges for an edge on a solid body
    Debug.Assert UBound(vCoEdgeArr) >= 0

    If 0 = UBound(vCoEdgeArr) Then
        Set swCoEdge = vCoEdgeArr(0)
        ' No ambiguity, so select
        Set swLoop = swCoEdge.GetLoop
        SelectLoop swApp, swModel, swLoop, swSelData
        Exit Sub
    End If

    ' 2: Coedges, so must have face to resolve ambiguity
    Debug.Assert Not swFace Is Nothing

    For Each vCoEdge In vCoEdgeArr
        Set swCoEdge = vCoEdge
        If swEdge Is swCoEdge.GetEdge Then
            Set swLoop = swCoEdge.GetLoop
            If swFace Is swLoop.GetFace Then
                SelectLoop swApp, swModel, swLoop, swSelData
            End If
        End If
    Next
End Sub
```

Select Multiple Objects for Selection Boxes Example (C#)

This example shows how to select multiple objects and add the selected objects to different selection boxes on a PropertyManager page.

```
//-----
// Preconditions:
// 1. Verify that the assembly to open exists.
// 2. Copy SolidWorksMacro.cs to the project.
// 3. Add a class module and copy clsPropMgr.cs to that module.
// 4. Add the SolidWorks.Interop.swpublished primary interop assembly
//    reference to your project (click Project > Add Reference,
//    browse to install_dir\api\redist, select
//    SolidWorks.Interop.swpublished.dll > OK).
// 5. Verify that the Tools > Options > System Options > Stop VSTA debugger
//    on macro exit checkbox is clear.
//
// Postconditions:
// 1. Opens the specified assembly.
// 2. Creates a PropertyManager page.
// 3. Examine the PropertyManager page to verify that three selected faces
//    appear in the top selection box, and two selected faces appear
//    in the bottom selection box.
// 4. Close the PropertyManager page.
//
// NOTE:
// * Reselect the Tools > Options > System Options > Stop VSTA debugger
//   on macro exit checkbox, if it was selected prior to running the macro.
// * Because the assembly document is used elsewhere,
//   do not save changes.
//-----
// SolidWorksMacro.cs

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;

namespace CreatePropertyManagerPageCSharp.csproj
{
    partial class SolidWorksMacro
    {
        public SldWorks swApp;
        public ModelDoc2 swModel;
        public ModelDocExtension swModelDocExt;
        public SelectionMgr swSelectionMgr;
        public const int mark = 1;
        public const int mark2 = 2;
        public clsPropMgr pm;

        public void Main()
        {
            int openDocErrors = 0;
            int openDocWarnings = 0;
            Face2[] selections1 = new Face2[3];
            Face2[] selections2 = new Face2[2];
            int i = 0;
            int j = 0;
            bool status = false;
            int nbrSelections = 0;

            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, false);

            string fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\advdrawings\\bladed shaft.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref o
penDocErrors, ref openDocWarnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;

            //Create a new instance of the PropertyManager class
            pm = new clsPropMgr(swApp);
            pm.Show();

            //Selections for top selection box
            status = swModelDocExt.SelectByID("", "FACE", 0.00369805475952489, 0.0901975482463513, 0.00315907187808762, true, mark, null, 0);
            status = swModelDocExt.SelectByID("", "FACE", 0.00860570843923369, 0.0737431971170679, 0.0039892160950501, true, mark, null, 0);
            status = swModelDocExt.SelectByID("", "FACE", 0.0100013029872912, 0.0446884591512742, 0.00996173127262523, true, mark, null, 0);
            nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1);
            nbrSelections = nbrSelections - 1;
            for (i = 0; i <= nbrSelections; i++)
            {
                selections1[i] = (Face2)swSelectionMgr.GetSelectedObject6(i + 1, -1);
            }

            swModel.ClearSelection2(true);

            //Selections for bottom selection box
            status = swModelDocExt.SelectByID("", "FACE", -0.0264206017159268, -0.00342602957275062, 0.00987615560137556, true, mark2, null, 0);
            status = swModelDocExt.SelectByID("", "FACE", -0.00128130257782288, -0.00354158999988385, 0.0177003152412567, true, mark2, null, 0);
            nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1);
            nbrSelections = nbrSelections - 1;
            for (j = 0; j <= nbrSelections; j++)
            {
                selections2[j] = (Face2)swSelectionMgr.GetSelectedObject6(j + 1, -1);
            }

            swModel.ClearSelection2(true);

            //Populate selection boxes
            SelectBoxFaces1(selections1, mark);
            SelectBoxFaces2(selections2, mark2);
        }
    }
}
```

```

    }
    private void SelectBoxFaces1(Face2[] selections, int selectionBoxMark)
    {
        SelectData swSelectData = default(SelectData);
        swSelectData = (SelectData)swSelectionMgr.CreateSelectData();
        swSelectData.Mark = selectionBoxMark;
        swModelDocExt.MultiSelect2(selections, true, swSelectData);
    }
    private void SelectBoxFaces2(Face2[] selections, int selectionBoxMark)
    {
        SelectData swSelectData = default(SelectData);
        swSelectData = (SelectData)swSelectionMgr.CreateSelectData();
        swSelectData.Mark = selectionBoxMark;
        swModelDocExt.MultiSelect2(selections, true, swSelectData);
    }
}
}

```

[Back to top](#)

// clsPropMgr.cs

```

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using SolidWorks.Interop.swpublished;
using System;
using System.Runtime.InteropServices;
using System.Diagnostics;

namespace CreatePropertyManagerPageCSharp.csproj
{
    [ComVisibleAttribute(true)]

    public class clsPropMgr : PropertyManagerPage2Handler9
    {
        //General objects required for the PropertyManager page
        PropertyManagerPage2 pm_Page;
        PropertyManagerPageGroup pm_Group;
        PropertyManagerPageSelectionbox pm_Selection;
        PropertyManagerPageSelectionbox pm_Selection2;

        //Each object in the page needs a unique ID
        const int GroupID = 1;
        const int SelectionID = 3;
        const int ComboID = 4;
        const int ListID = 5;
        const int Selection2ID = 6;

        public void Show()
        {
            pm_Page.Show2(0);
        }

        //The following runs when a new instance
        //of the class is created
        public clsPropMgr(SldWorks swApp)
        {
            string PageTitle = null;
            string caption = null;
            string tip = null;
            long options = 0;
            int longererrors = 0;
            int controlType = 0;
            int alignment = 0;

            //Set the variables for the page
            PageTitle = "MultiSelect2 Test";
            options = (int)swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton + (int)swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton + (int)swPropertyManagerPageOptions_e.swPropertyManagerOptions_PushpinButton;

            //Create the PropertyManager page
            pm_Page = (PropertyManagerPage2)swApp.CreatePropertyManagerPage(PageTitle, (int)options, this, ref longererrors);

            //Make sure that the page was created properly
            if (longererrors == (int)swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay)
            {
                //Begin adding the controls to the page

                //Create the group box
                caption = "";
                options = (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + (int)swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded;
                pm_Group = (PropertyManagerPageGroup)pm_Page.AddGroupBox(GroupID, caption, (int)options);

                //Create two selection boxes
                controlType = (int)swPropertyManagerPageControlType_e.swControlType_Selectionbox;
                caption = "";
                alignment = (int)swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent;
                options = (int)swAddControlOptions_e.swControlOptions_Visible + (int)swAddControlOptions_e.swControlOptions_Enabled;
                tip = "Select multiple faces.";

                pm_Selection = (PropertyManagerPageSelectionbox)pm_Group.AddControl(SelectionID, (short)controlType, caption, (short)alignment, (int)options, tip);
                pm_Selection2 = (PropertyManagerPageSelectionbox)pm_Group.AddControl(Selection2ID, (short)controlType, caption, (short)alignment, (int)options, tip);

                'Only faces can populate the selection boxes
                swSelectType_e[] filters = new swSelectType_e[1];
                filters[0] = swSelectType_e.swSelFACES;

                object filterObj = null;
                filterObj = filters;

                pm_Selection.SingleEntityOnly = false;
                pm_Selection.AllowMultipleSelectOfSameEntity = true;
                pm_Selection.Height = 50;
                pm_Selection.SetSelectionFilters(filterObj);
                pm_Selection.Mark = SolidWorksMacro.mark;

                pm_Selection2.SingleEntityOnly = false;
                pm_Selection2.AllowMultipleSelectOfSameEntity = true;
                pm_Selection2.Height = 50;
                pm_Selection2.SetSelectionFilters(filterObj);
                pm_Selection2.Mark = SolidWorksMacro.mark2;
            }
        }
    }
}

```

```

        else
        {
            //If the page is not created
            System.Windows.Forms.MessageBox.Show("An error occurred while attempting to create the " + "PropertyManager Page");
        }
    }
}

#region IPropertyManagerPage2Handler9 Members
void IPropertyManagerPage2Handler9.AfterActivation()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.AfterClose()
{
    throw new Exception("The method or operation is not implemented.");
}

int IPropertyManagerPage2Handler9.OnActiveXControlCreated(int Id, bool Status)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnButtonPress(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnCheckboxCheck(int Id, bool Checked)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnClose(int Reason)
{
    if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel)
    {
        //Do something when the cancel button is clicked
    }
    else if (Reason == (int)swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay)
    {
        //Do something else when the OK button is clicked
    }
}

void IPropertyManagerPage2Handler9.OnComboboxEditChanged(int Id, string Text)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnComboboxSelectionChanged(int Id, int Item)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnGainedFocus(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnGroupCheck(int Id, bool Checked)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnGroupExpand(int Id, bool Expanded)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnHelp()
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnKeystroke(int Wparam, int Message, int Lparam, int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnListboxSelectionChanged(int Id, int Item)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnLostFocus(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnNextPage()

```

```

{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnNumberboxChanged(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnOptionCheck(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnPopupMenuItem(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnPopupMenuItemUpdate(int Id, ref int retval)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnPreview()
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnPreviousPage()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnRedo()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxCalloutCreated(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSelectionboxListChanged(int Id, int Count)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSliderPositionChanged(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnSliderTrackingCompleted(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

bool IPropertyManagerPage2Handler9.OnSubmitSelection(int Id, object Selection, int SelType, ref string ItemText)
{
    // This method must return true for selections to occur
    Debug.Print("OnSubmitSelection fired.");
    return true;
}

bool IPropertyManagerPage2Handler9.OnTabClicked(int Id)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnTextboxChanged(int Id, string Text)
{
    throw new Exception("The method or operation is not implemented.");
}

```

```

void IPropertyManagerPage2Handler9.OnUndo()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnWhatsNew()
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnListboxRMBUp(int Id, int PosX, int PosY)
{
    throw new Exception("The method or operation is not implemented.");
}

int IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated(int Id, bool Status)
{
    throw new Exception("The method or operation is not implemented.");
}

void IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted(int Id, double Value)
{
    throw new Exception("The method or operation is not implemented.");
}

#endregion
}
}

```

[Back to top](#)

Select Multiple Objects for Selection Boxes Example (VBA)

This example shows how to select multiple objects and add the selected objects to different selection boxes on a PropertyManager page.

```

'-----
' Preconditions:
' 1. Copy Modules - main to your project.
' 2. Insert a class module and copy Class Modules - clsPropMgr to that module.
' 3. Rename the class module to clsPropMgr.
' 3. Add swpublished.tlb to your project (click Tools > References > SOLIDWORKS version exposed type libraries for add-in use).
' 4. Verify that the specified assembly exists.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates a PropertyManager page.
' 3. Examine the PropertyManager page and verify that three selected faces
'    appear in the top selection box and two selected faces appear in the
'    bottom selection box.
' 4. Close the PropertyManager page.
'
' NOTE: Because the assembly document is used elsewhere,
' do not save changes.
'-----

'Modules - main

Option Explicit

Public swApp As SldWorks.SldWorks
Public swModel As SldWorks.ModelDoc2
Public swModelDocExt As SldWorks.ModelDocExtension
Public swSelectionMgr As SldWorks.SelectionMgr
Public pm As clsPropMgr
Public mark As Long
Public mark2 As Long

Sub main()

    Dim openDocErrors As Long
    Dim openDocWarnings As Long
    Dim nbrSelections As Long
    Dim selections1(2) As SldWorks.Face2
    Dim selections2(1) As SldWorks.Face2
    Dim i As Long
    Dim j As Long
    Dim status As Boolean
    Dim fileName As String

    'Set the marks for the selections and selection boxes
    mark = 1
    mark2 = 2

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocASSEMBLY, swOpenDocOptions_Silent, "", openDocErrors, openDocWarnings)
    Set swModelDocExt = swModel.Extension
    Set swSelectionMgr = swModel.SelectionManager

    'Create a new instance of the PropertyManager class
    Set pm = New clsPropMgr
    pm.Show

    'Selections for top selection box
    status = swModelDocExt.SelectByID("", "FACE", 3.69805475952489E-03, 9.01975482463513E-02, 3.15907187808762E-03, True, mark, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", 8.60570843923369E-03, 7.37431971170679E-02, 3.9892160950501E-03, True, mark, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", 1.00013029872912E-02, 4.46884591512742E-02, 9.96173127262523E-03, True, mark, Nothing, 0)
    nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
    nbrSelections = nbrSelections - 1
    For i = 0 To nbrSelections
        Set selections1(i) = swSelectionMgr.GetSelectedObject6(i + 1, -1)
    Next

    swModel.ClearSelection2 True

    'Selections for bottom selection box
    status = swModelDocExt.SelectByID("", "FACE", -2.64206017159268E-02, -3.42602957275062E-03, 9.87615560137556E-03, True, mark2, Nothing, 0)
    status = swModelDocExt.SelectByID("", "FACE", -1.28130257782288E-03, -3.5415899988385E-03, 1.77003152412567E-02, True, mark2, Nothing, 0)
    nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
    nbrSelections = nbrSelections - 1
    For j = 0 To nbrSelections
        Set selections2(j) = swSelectionMgr.GetSelectedObject6(j + 1, -1)
    Next

    swModel.ClearSelection2 True

    'Populate selection boxes
    SelectBoxFaces1 selections1, mark

    SelectBoxFaces2 selections2, mark2

End Sub

```

```

Private Sub SelectBoxFaces1(selections() As SldWorks.Face2, selectionBoxMark)

    Dim swSelectData As SldWorks.SelectData
    Set swSelectData = swSelectionMgr.CreateSelectData
    swSelectData.mark = selectionBoxMark
    swModelDocExt.MultiSelect2 selections, True, swSelectData

End Sub

Private Sub SelectBoxFaces2(selections() As SldWorks.Face2, selectionBoxMark)

    Dim swSelectData As SldWorks.SelectData
    Set swSelectData = swSelectionMgr.CreateSelectData
    swSelectData.mark = selectionBoxMark
    swModelDocExt.MultiSelect2 selections, True, swSelectData

End Sub

```

[Back to top](#)

'Class Modules - clsPropMgr

Option Explicit

'Required for PropertyManager page controls
Implements PropertyManagerPage2Handler9

'General objects required for the PropertyManager page
Dim pm_Page As PropertyManagerPage2
Dim pm_Group As PropertyManagerPageGroup
Dim pm_Selection As PropertyManagerPageSelectionbox
Dim pm_Selection2 As PropertyManagerPageSelectionbox

'Each object in the PropertyManager page needs a unique ID
Const GroupID As Long = 1
Const LabelID As Long = 2
Const SelectionID As Long = 3
Const ComboID As Long = 4
Const ListID As Long = 5
Const Selection2ID As Long = 6

Dim ClickedCancel As Boolean
Dim retVal As Long

Sub Show()
 pm_Page.Show2 0
End Sub

'The following runs when a new instance
'of the class is created
Private Sub Class_Initialize()

Dim PageTitle As String
Dim caption As String
Dim tip As String
Dim options As Long
Dim longerrors As Long
Dim controlType As Long
Dim alignment As Long

'Set the variables for the PropertyManager page
PageTitle = "MultiSelect2 Test"
options = swPropertyManager_OkayButton + swPropertyManager_CancelButton + swPropertyManagerOptions_PushpinButton

'Create the PropertyManager page
Set pm_Page = swApp.CreatePropertyManagerPage(PageTitle, options, Me, longerrors)

'Make sure that the PropertyManager page was created properly
If longerrors = swPropertyManagerPage_Okay Then

'Begin adding the controls to the PropertyManager page

'Create the group box
 caption = ""
 options = swGroupBoxOptions_Visible + swGroupBoxOptions_Expanded

Set pm_Group = pm_Page.AddGroupBox(GroupID, caption, options)

'Create selection boxes
 controlType = swControlType_Selectionbox
 caption = "" ' No caption for selection box
 alignment = swControlAlign_Indent
 options = swControlOptions_Visible + swControlOptions_Enabled
 tip = "Select multiple faces."

Set pm_Selection = pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip)
 Set pm_Selection2 = pm_Group.AddControl(Selection2ID, controlType, caption, alignment, options, tip)

'Only faces can populate the selection boxes
 Dim filters(0) As Long
 filters(0) = swSelFACES

pm_Selection.SingleEntityOnly = False
 pm_Selection.AllowMultipleSelectOfSameEntity = True
 pm_Selection.Height = 50
 pm_Selection.SetSelectionFilters filters
 pm_Selection.mark = mark

pm_Selection2.SingleEntityOnly = False
 pm_Selection2.AllowMultipleSelectOfSameEntity = True
 pm_Selection2.Height = 50

```

    pm_Selection2.SetSelectionFilters filters
    pm_Selection2.mark = mark2

Else 'If the page is not created
    MsgBox "An error occurred while attempting to create the " & "PropertyManager Page", vbCritical

End If

End Sub

Private Sub PropertyManagerPage2Handler9_AfterActivation()

End Sub

Private Sub PropertyManagerPage2Handler9_AfterClose()
'Destroy the class
Set pm = Nothing
End Sub

Private Function PropertyManagerPage2Handler9_OnActiveXControlCreated(ByVal Id As Long, ByVal status As Boolean) As Long

End Function

Private Sub PropertyManagerPage2Handler9_OnButtonPress(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler9_OnCheckboxCheck(ByVal Id As Long, ByVal Checked As Boolean)

End Sub

Private Sub PropertyManagerPage2Handler9_OnClose(ByVal Reason As Long)

If Reason = swPropertyManagerPageClose_Cancel Then
    'Cancel button was clicked
    ClickedCancel = True
ElseIf Reason = swPropertyManagerPageClose_Okay Then
    'OK button was clicked
    ClickedCancel = False
End If

End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxEditChanged(ByVal Id As Long, ByVal Text As String)

End Sub

Private Sub PropertyManagerPage2Handler9_OnComboboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)

End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupCheck(ByVal Id As Long, ByVal Checked As Boolean)

End Sub

Private Sub PropertyManagerPage2Handler9_OnGroupExpand(ByVal Id As Long, ByVal Expanded As Boolean)

End Sub

Private Function PropertyManagerPage2Handler9_OnHelp() As Boolean

End Function

Private Function PropertyManagerPage2Handler9_OnKeystroke(ByVal Wparam As Long, ByVal Message As Long, ByVal Lparam As Long, ByVal Id As Long) As Boolean

End Function

Private Sub PropertyManagerPage2Handler9_OnListboxSelectionChanged(ByVal Id As Long, ByVal Item As Long)

End Sub

Private Function PropertyManagerPage2Handler9_OnNextPage() As Boolean

End Function

Private Sub PropertyManagerPage2Handler9_OnNumberboxChanged(ByVal Id As Long, ByVal Value As Double)

End Sub

Private Sub PropertyManagerPage2Handler9_OnOptionCheck(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItem(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler9_OnPopupMenuItemUpdate(ByVal Id As Long, retVal As Long)

End Sub

Private Function PropertyManagerPage2Handler9_OnPreview() As Boolean

End Function

Private Function PropertyManagerPage2Handler9_OnPreviousPage() As Boolean

End Function

Private Sub PropertyManagerPage2Handler9_OnRedo()

End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutCreated(ByVal Id As Long)

End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxCalloutDestroyed(ByVal Id As Long)

End Sub

```

```

Private Sub PropertyManagerPage2Handler9_OnSelectionboxFocusChanged(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSelectionboxListChanged(ByVal Id As Long, ByVal Count As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderPositionChanged(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Sub PropertyManagerPage2Handler9_OnSliderTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

Private Function PropertyManagerPage2Handler9_OnSubmitSelection(ByVal Id As Long, ByVal Selection As Object, ByVal SelType As Long, ItemText As String) As Boolean
    PropertyManagerPage2Handler9_OnSubmitSelection = True
    Debug.Print ("OnSubmitSelection event fired.")
End Function

Private Function PropertyManagerPage2Handler9_OnTabClicked(ByVal Id As Long) As Boolean
End Function

Private Sub PropertyManagerPage2Handler9_OnTextboxChanged(ByVal Id As Long, ByVal Text As String)
End Sub

Private Sub PropertyManagerPage2Handler9_OnUndo()
End Sub

Private Sub PropertyManagerPage2Handler9_OnWhatsNew()
End Sub

Private Sub PropertyManagerPage2Handler9_OnLostFocus(ByVal Id As Long)
End Sub

Private Sub PropertyManagerPage2Handler9_OnGainedFocus(ByVal Id As Long)

End Sub

Public Sub PropertyManagerPage2Handler9_OnListBoxRMBUp(ByVal Id As Long, ByVal posX As Long, ByVal posY As Long)
End Sub

Public Function PropertyManagerPage2Handler9_OnWindowFromHandleControlCreated(ByVal Id As Long, ByVal status As Boolean) As Long
End Function

Public Sub PropertyManagerPage2Handler9_OnNumberboxTrackingCompleted(ByVal Id As Long, ByVal Value As Double)
End Sub

```

[Back to top](#)

Select Multiple Objects for Selection Boxes Example (VB.NET)

This example shows how to select multiple objects and add the selected objects to different selection boxes on a PropertyManager page.

```

'-----
' Preconditions:
' 1. Verify that the specified assembly exists.
' 2. Copy SolidWorksMacro.vb to your project.
' 3. Add a class module and copy clsPropMgr.vb to that module.
' 4. Add the SolidWorks.Interop.swpublished primary interop assembly
'    reference to your project (click Project > Add Reference,
'    browse to install_dir\api\redist, select
'    SolidWorks.Interop.swpublished.dll > OK).
' 5. Verify that the Tools > Options > System Options > Stop VSTA debugger
'    on macro exit checkbox is clear.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Creates a PropertyManager page.
' 3. Examine the PropertyManager page to verify that
'    three selected faces appear in the top selection
'    box and two selected faces appear in the bottom
'    selection box.
' 4. Close the PropertyManager page.
'
' NOTES:
' * Reselect the Tools > Options > System Options > Stop VSTA debugger
'   on macro exit checkbox, if it was selected prior to running the macro.
' * Because the assembly document is used elsewhere,
'   do not save changes.
'-----

'SolidWorksMacro.vb

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro

    Public swApp As SldWorks
    Public swModel As ModelDoc2
    Public swModelDocExt As ModelDocExtension
    Public swSelectionMgr As SelectionMgr
    Public WithEvents pm As clsPropMgr
    Public Const mark As Integer = 1
    Public Const mark2 As Integer = 2

    Public Sub Main()

        Dim openDocErrors As Integer
        Dim openDocWarnings As Integer
        Dim fileName As String
        Dim selections1(2) As Face2
        Dim selections2(1) As Face2
        Dim i As Integer
        Dim j As Integer
        Dim status As Boolean
        Dim nbrSelections As Integer

        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\advdrawings\bladed shaft.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", openDocErrors, openDocWarnings)

        swModelDocExt = swModel.Extension
        swSelectionMgr = swModel.SelectionManager

        'Create a new instance of the PropertyManager class
        pm = New clsPropMgr(swApp)
        pm.Show()

        'Selections for top selection box
        status = swModelDocExt.SelectByID2("", "FACE", 0.00369805475952489, 0.0901975482463513, 0.00315907187808762, True, mark, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.00860570843923369, 0.0737431971170679, 0.0039892160950501, True, mark, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", 0.0100013029872912, 0.0446884591512742, 0.00996173127262523, True, mark, Nothing, 0)
        nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
        nbrSelections = nbrSelections - 1
        For i = 0 To nbrSelections
            selections1(i) = swSelectionMgr.GetSelectedObject6(i + 1, -1)
        Next

        swModel.ClearSelection2(True)

        'Selections for bottom selection box
        status = swModelDocExt.SelectByID2("", "FACE", -0.0264206017159268, -0.00342602957275062, 0.00987615560137556, True, mark2, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "FACE", -0.00128130257782288, -0.00354158999988385, 0.0177003152412567, True, mark2, Nothing, 0)
        nbrSelections = swSelectionMgr.GetSelectedObjectCount2(-1)
        nbrSelections = nbrSelections - 1
        For j = 0 To nbrSelections
            selections2(j) = swSelectionMgr.GetSelectedObject6(j + 1, -1)
        Next

        swModel.ClearSelection2(True)

        'Populate selection boxes
        SelectBoxFaces1(selections1, mark)
        SelectBoxFaces2(selections2, mark2)

    End Sub

    Private Sub SelectBoxFaces1(ByVal selections() As Face2, ByVal selectionBoxMark As Integer)
        Dim swSelectData As SelectData
        swSelectData = swSelectionMgr.CreateSelectData
        swSelectData.Mark = selectionBoxMark
    End Sub

```

```

        swModelDocExt.MultiSelect2(selections, True, swSelectData)
    End Sub
    Private Sub SelectBoxFaces2(ByVal selections() As Face2, ByVal selectionBoxMark As Integer)
        Dim swSelectData As SelectData
        swSelectData = swSelectionMgr.CreateSelectData
        swSelectData.Mark = selectionBoxMark
        swModelDocExt.MultiSelect2(selections, True, swSelectData)
    End Sub
End Class

```

[Back to top](#)

'clsPropMgr.vb

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports SolidWorks.Interop.swpublished
Imports System
Imports System.Runtime.InteropServices
Imports System.Diagnostics

<ComVisibleAttribute(True)> Public Class clsPropMgr

    Implements IPropertyManagerPage2Handler9

    Dim swApp As SldWorks

    'General objects required for the PropertyManager page
    Dim pm_Page As PropertyManagerPage2
    Dim pm_Group As PropertyManagerPageGroup
    Dim pm_Selection As PropertyManagerPageSelectionbox
    Dim pm_Selection2 As PropertyManagerPageSelectionbox

    'Each object in the page needs a unique ID
    Const GroupID As Integer = 1
    Const LabelID As Integer = 2
    Const SelectionID As Integer = 3
    Const ComboID As Integer = 4
    Const ListID As Integer = 5
    Const Selection2ID As Integer = 6

    Dim ClickedCancel As Boolean
    Dim retVal As Integer

    Sub Show()
        pm_Page.Show2(0)
    End Sub

    'The following runs when a new instance
    'of the class is created
    Public Sub New(ByVal swApp As SldWorks)

        Dim PageTitle As String
        Dim caption As String
        Dim tip As String
        Dim options As Long
        Dim longerrors As Long
        Dim controlType As Integer
        Dim alignment As Integer

        'Set the variables for the page
        PageTitle = "MultiSelect2 Test"
        options = swPropertyManagerButtonTypes_e.swPropertyManager_OkayButton + swPropertyManagerButtonTypes_e.swPropertyManager_CancelButton + swProperty
        ManagerPageOptions_e.swPropertyManagerOptions_PushpinButton

        'Create the PropertyManager page
        pm_Page = CType(swApp.CreatePropertyManagerPage(PageTitle, options, Me, longerrors), PropertyManagerPage2)

        'Make sure that the page was created properly
        If longerrors = swPropertyManagerPageStatus_e.swPropertyManagerPage_Okay Then

            'Begin adding the controls to the page

            'Create the group box
            caption = ""
            options = swAddGroupBoxOptions_e.swGroupBoxOptions_Visible + swAddGroupBoxOptions_e.swGroupBoxOptions_Expanded
            pm_Group = pm_Page.AddGroupBox(GroupID, caption, options)

            'Create two selection boxes
            controlType = swPropertyManagerPageControlType_e.swControlType_Selectionbox
            caption = "" ' No caption for selection boxes
            alignment = swPropertyManagerPageControlLeftAlign_e.swControlAlign_Indent
            options = swAddControlOptions_e.swControlOptions_Visible + swAddControlOptions_e.swControlOptions_Enabled
            tip = "Select multiple faces."

            pm_Selection = pm_Group.AddControl(SelectionID, controlType, caption, alignment, options, tip)
            pm_Selection2 = pm_Group.AddControl(Selection2ID, controlType, caption, alignment, options, tip)

            'Only faces can populate the selection boxes
            Dim filters(0) As swSelectType_e
            filters(0) = swSelectType_e.swSelFACES

            Dim filterObj As Object
            filterObj = filters

            pm_Selection.SingleEntityOnly = False
            pm_Selection.AllowMultipleSelectOfSameEntity = True
            pm_Selection.Height = 50
            pm_Selection.SetSelectionFilters(filterObj)
            pm_Selection.Mark = SolidWorksMacro.mark

            pm_Selection2.SingleEntityOnly = False
            pm_Selection2.AllowMultipleSelectOfSameEntity = True
            pm_Selection2.Height = 50
            pm_Selection2.SetSelectionFilters(filterObj)
            pm_Selection2.Mark = SolidWorksMacro.mark2

        Else 'If the page is not created
            MsgBox("An error occurred while attempting to create the " & "PropertyManager Page", vbCritical)
        End If

    End Sub

    Public Sub AfterActivation() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterActivation

    End Sub

```

```

Public Sub AfterClose() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.AfterClose
End Sub

Public Function OnActiveXControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnActiveXControlCreated
End Function

Public Sub OnButtonPress(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnButtonPress
End Sub

Public Sub OnCheckboxCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnCheckboxCheck
End Sub

Public Sub OnClose(ByVal Reason As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnClose
    If Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Cancel Then
        'Cancel button was clicked
        ClickedCancel = True
    ElseIf Reason = swPropertyManagerPageCloseReasons_e.swPropertyManagerPageClose_Okay Then
        'OK button was clicked
        ClickedCancel = False
    End If
End Sub

Public Sub OnComboboxEditChanged(ByVal Id As Integer, ByVal Text As String) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnComboboxEditChanged
End Sub

Public Sub OnComboboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnComboboxSelectionChanged
End Sub

Public Sub OnGainedFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnGainedFocus
End Sub

Public Sub OnGroupCheck(ByVal Id As Integer, ByVal Checked As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnGroupCheck
End Sub

Public Sub OnGroupExpand(ByVal Id As Integer, ByVal Expanded As Boolean) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnGroupExpand
End Sub

Public Function OnHelp() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnHelp
End Function

Public Function OnKeystroke(ByVal Wparam As Integer, ByVal Message As Integer, ByVal Lparam As Integer, ByVal Id As Integer) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnKeystroke
End Function

Public Sub OnListboxSelectionChanged(ByVal Id As Integer, ByVal Item As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnListboxSelectionChanged
End Sub

Public Sub OnLostFocus(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnLostFocus
End Sub

Public Function OnNextPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNextPage
End Function

Public Sub OnNumberboxChanged(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNumberboxChanged
End Sub

Public Sub OnOptionCheck(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnOptionCheck
End Sub

Public Sub OnPopupMenuItem(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPopupMenuItem
End Sub

Public Sub OnPopupMenuItemUpdate(ByVal Id As Integer, ByRef retval As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPopupMenuItemUpdate
End Sub

Public Function OnPreview() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPreview
End Function

Public Function OnPreviousPage() As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnPreviousPage
End Function

Public Sub OnRedo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnRedo
End Sub

Public Sub OnSelectionboxCalloutCreated(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxCalloutCreated
End Sub

Public Sub OnSelectionboxCalloutDestroyed(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxCalloutDestroyed
End Sub

Public Sub OnSelectionboxFocusChanged(ByVal Id As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxFocusChanged

```

```

End Sub

Public Sub OnSelectionboxListChanged(ByVal Id As Integer, ByVal Count As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSelectionboxListChanged

End Sub

Public Sub OnSliderPositionChanged(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderPositionChanged

End Sub

Public Sub OnSliderTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSliderTrackingCompleted

End Sub

Public Function OnSubmitSelection(ByVal Id As Integer, ByVal Selection As Object, ByVal SelType As Integer, ByRef ItemText As String) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnSubmitSelection
    OnSubmitSelection = True
    Debug.Print("OnSubmitSelection fired.")
End Function

Public Function OnTabClicked(ByVal Id As Integer) As Boolean Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTabClicked

End Function

Public Sub OnTextboxChanged(ByVal Id As Integer, ByVal Text As String) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnTextboxChanged

End Sub

Public Sub OnUndo() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnUndo

End Sub

Public Sub OnWhatsNew() Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWhatsNew

End Sub

Public Sub OnListBoxRMBUp(ByVal Id As Integer, ByVal posX As Integer, ByVal posY As Integer) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnListBoxRMBUp

End Sub

Public Function OnWindowFromHandleControlCreated(ByVal Id As Integer, ByVal Status As Boolean) As Integer Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnWindowFromHandleControlCreated

End Function

Public Sub OnNumberBoxTrackingCompleted(ByVal Id As Integer, ByVal Value As Double) Implements SolidWorks.Interop.swpublished.IPropertyManagerPage2Handler9.OnNumberBoxTrackingCompleted

End Sub

End Class

```

[Back to top](#)

Select Multiple Sketch Segments for Sweep Path Example (C#)

This example shows how to select multiple sketch segments for the path for a sweep feature.

```
//-----
// Preconditions: Verify that the part template exists.
//
// Postconditions:
// 1. Opens a new part.
// 2. Creates:
//    * sketch of a circle.
//    * sketch of a line.
//    * another sketch of a line.
// 3. Selects the sketch of the circle for the sweep profile.
// 4. Selects the sketches of the lines for the sweep path
//    and groups them as an object.
// 5. Creates a sweep feature
// 6. Examine the FeatureManager design tree and graphics
//    area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            bool status = false;

            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0, 0);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swSketchManager = (SketchManager)swModel.SketchManager;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;

            //Create sketch of circle for the sweep profile
            swSketchSegment = (SketchSegment)swSketchManager.CreateCircle(0.0, 0.0, 0.0, 0.002394, -0.006333, 0.0);
            swSketchManager.InsertSketch(true);

            //Create sketches of lines for the sweep path
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(-0.0, 0.0, 0.0, 0.088481, 0.035691, 0.0);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchManager.InsertSketch(true);
            swSketchSegment = (SketchSegment)swSketchManager.CreateLine(0.088481, 0.035691, 0.0, 0.079214, 0.076295, 0.0);
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Select the sketch of the circle for the sweep profile
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", -0.00586834883582351, -0.00337646707201764, 0, false, 1, null, 0);

            //Select the sketches of the lines for the sweep path and group them as an object
            status = swModelDocExt.SelectByID2("Line1@Sketch2", "EXTSKETCHSEGMENT", 0.0379259971310087, 0.0152983890733924, 0, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("Line1@Sketch3", "EXTSKETCHSEGMENT", 0.0848435978763939, 0.0516285284155501, 0, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("Unknown", "SELOBJGROUP", 0, 0, 0, true, 4, null, 0);

            //Create the sweep feature
            swFeature = (Feature)swFeatureManager.InsertProtrusionSwept4(false, false, 0, false, false, 0, 0, false, 0, 0,
            0, 0, true, true, true, 0, true, false, 0, 0);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Select Multiple Sketch Segments for Sweep Path Example (VBA)

This example shows how to select multiple sketch segments for the path for a sweep feature.

```
'-----
' Preconditions: Verify that the part template exists.
'
' Postconditions:
' 1. Opens a new part.
' 2. Creates:
'   * sketch of a circle.
'   * sketch of a line.
'   * another sketch of a line.
' 3. Selects the sketch of the circle for the sweep profile.
' 4. Selects the sketches of the lines for the sweep path
'   and groups them as an object.
' 5. Creates a sweep feature.
' 6. Examine the FeatureManager design tree and graphics
'   area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketchManager As SldWorks.SketchManager
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim status As Boolean

Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
    Set swModelDocExt = swModel.Extension
    Set swSketchManager = swModel.SketchManager
    Set swFeatureManager = swModel.FeatureManager

    'Create sketch of circle for the sweep profile
    Set swSketchSegment = swSketchManager.CreateCircle(0#, 0#, 0#, 0.002394, -0.006333, 0#)
    swSketchManager.InsertSketch True

    'Create sketches of lines for the sweep path
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateLine(-0#, 0#, 0#, 0.088481, 0.035691, 0#)
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swSketchManager.InsertSketch True
    Set swSketchSegment = swSketchManager.CreateLine(0.088481, 0.035691, 0#, 0.079214, 0.076295, 0#)
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    'Select the sketch of the circle for the sweep profile
    status = swModelDocExt.SelectByID("Sketch1", "SKETCH", -5.86834883582351E-03, -3.37646707201764E-03, 0, False, 1, Nothing, 0)

    'Select the sketches of the lines for the sweep path and group them as an object
    status = swModelDocExt.SelectByID("Line1@Sketch2", "EXTSKETCHSEGMENT", 3.79259971310087E-02, 1.52983890733924E-02, 0, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("Line1@Sketch3", "EXTSKETCHSEGMENT", 8.48435978763939E-02, 5.16285284155501E-02, 0, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID("Unknown", "SELOBJGROUP", 0, 0, 0, True, 4, Nothing, 0)

    'Create the sweep feature
    Set swFeature = swFeatureManager.InsertProtrusionSwept4(False, False, 0, False, False, 0, 0, False, 0, 0, 0, 0, True, True, True, 0, True, False, 0,
0)

End Sub
```

Select Multiple Sketch Segments for Sweep Path Example (VB.NET)

This example shows how to select multiple sketch segments for the path for a sweep feature.

```

-----
' Preconditions: Verify that the part template exists.
-----
' Postconditions:
' 1. Opens a new part.
' 2. Creates:
'    * sketch of a circle.
'    * sketch of a line.
'    * another sketch of a line.
' 3. Selects the sketch of the circle for the sweep profile.
' 4. Selects the sketches of the lines for the sweep path
'    and groups them as an object.
' 5. Creates a sweep feature.
' 6. Examine the FeatureManager design tree and graphics
'    area.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchSegment As SketchSegment
        Dim swSketchManager As SketchManager
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim status As Boolean

        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prt", 0, 0, 0)
        swModelDocExt = swModel.Extension
        swSketchManager = swModel.SketchManager
        swFeatureManager = swModel.FeatureManager

        'Create sketch of circle for the sweep profile
        swSketchSegment = swSketchManager.CreateCircle(0.0#, 0.0#, 0.0#, 0.002394, -0.006333, 0.0#)
        swSketchManager.InsertSketch(True)

        'Create sketches of lines for the sweep path
        status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(-0.0#, 0.0#, 0.0#, 0.088481, 0.035691, 0.0#)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
        swSketchManager.InsertSketch(True)
        swSketchSegment = swSketchManager.CreateLine(0.088481, 0.035691, 0.0#, 0.079214, 0.076295, 0.0#)
        swSketchManager.InsertSketch(True)
        swModel.ClearSelection2(True)

        'Select the sketch of the circle for the sweep profile
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", -0.00586834883582351, -0.00337646707201764, 0, False, 1, Nothing, 0)

        'Select the sketches of the lines for the sweep path and group them as an object
        status = swModelDocExt.SelectByID("Line1@Sketch2", "EXTSKETCHSEGMENT", 0.0379259971310087, 0.0152983890733924, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("Line1@Sketch3", "EXTSKETCHSEGMENT", 0.0848435978763939, 0.0516285284155501, 0, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID("Unknown", "SELOBJGROUP", 0, 0, 0, True, 4, Nothing, 0)

        'Create the sweep feature
        swFeature = swFeatureManager.InsertProtrusionSwept4(False, False, 0, False, False, 0, 0, False, 0, 0, 0, 0, True, True, True, 0, True, False, 0, 0)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Select Multiple Splines for Loft Guide Curves Example (C#)

This example shows how to select multiple splines for the guide curves for a loft feature.

```
//-----
// Preconditions: Verify that the specified part template exists.
//
// Postconditions:
// 1. Creates a new part.
// 2. Creates a profile sketch.
// 3. Creates a reference plane and another profile sketch on that
//    reference plane.
// 4. Creates two splines for the guide curves.
// 5. Selects the profile sketches.
// 6. Selects the splines and groups them as an object.
// 7. Creates a loft feature.
// 8. Examine the FeatureManager design tree and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            SketchSegment swSketchSegment = default(SketchSegment);
            SketchManager swSketchManager = default(SketchManager);
            RefPlane swRefPlane = default(RefPlane);
            FeatureManager swFeatureManager = default(FeatureManager);
            bool status = false;

            //Create a new part
            swModel = (ModelDoc2)swApp.NewDocument("C:\\ProgramData\\SolidWorks\\SOLIDWORKS 2017\\templates\\Part.prtdot", 0, 0, 0);

            //Create a profile sketch
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            swModel.ClearSelection2(true);
            swSketchManager = (SketchManager)swModel.SketchManager;
            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0, 0, 0, 0.0706113079019074, 0, 0, 0, 0.0374944141689373, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Create a reference plane and another profile sketch
            //on that reference plane
            status = swModelDocExt.SelectByID2("Front Plane", "PLANE", 0, 0, 0, true, 0, null, 0);
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swRefPlane = (RefPlane)swFeatureManager.InsertRefPlane(8, 0.07, 0, 0, 0, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, false, 0, null, 0);
            swSketchSegment = (SketchSegment)swSketchManager.CreateEllipse(0, 0, 0, 0.0527205722070845, 0, 0, 0, 0.0154164850136235, 0);
            swModel.ClearSelection2(true);
            swSketchManager.InsertSketch(true);

            //Create a spline
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            object pointArray = null;
            double[] points = new double[15];
            points[0] = -0.07;
            points[1] = 0.0154164850136235;
            points[2] = 0;
            points[3] = -0.0531092941649547;
            points[4] = 0.0280386111480766;
            points[5] = 0;
            points[6] = -0.0296934467839947;
            points[7] = 0.0229795168190776;
            points[8] = 0;
            points[9] = -0.0112921067380967;
            points[10] = 0.026354325474415;
            points[11] = 0;
            points[12] = 0;
            points[13] = 0.0374944141689373;
            points[14] = 0;
            pointArray = points;
            swSketchSegment = (SketchSegment)swSketchManager.CreateSpline((pointArray));
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Create another spline
            status = swModelDocExt.SelectByID2("Right Plane", "PLANE", 0, 0, 0, false, 0, null, 0);
            points = new double[9];
            points[0] = -0.07;
            points[1] = -0.0154164850136235;
            points[2] = 0;
            points[3] = -0.0307689275649068;
            points[4] = -0.0233694015292372;
            points[5] = 0;
            points[6] = 0;
            points[7] = -0.0374944141689373;
            points[8] = 0;
            pointArray = points;
            swSketchSegment = swSketchManager.CreateSpline((pointArray));
            swSketchManager.InsertSketch(true);
            swModel.ClearSelection2(true);

            //Select the profile sketches
            status = swModelDocExt.SelectByID2("Sketch1", "SKETCH", -0.0585496337278505, 0.0209585732143712, 1, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Sketch2", "SKETCH", -0.0379093739088495, 0.0107136192740755, 1, true, 0, null, 0);

            //Select the splines for the guide curves
            status = swModelDocExt.SelectByID2("Spline1@Sketch3", "EXTSKETCHSEGMENT", -0.00620659823337474, 0.0304187689522769, 2, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("Spline1@Sketch4", "EXTSKETCHSEGMENT", -0.0402947949143199, -0.0206106896601265, 2, true, 0, null, 0);
            //Group the selected splines as an object
            status = swModelDocExt.SelectByID2("Unknown", "SELOBJGROUP", 0, 0, 0, true, 2, null, 0);
        }
    }
}
```

```
        //Create a loft
        swFeatureManager.InsertProtrusionBlend2(false, true, false, 1, 0, 0, 1, 1, true, true,
        false, 0, 0, 0, true, true, true, 0);

    }

    /// <summary>
    /// The SldWorks swApp variable is pre-assigned for you.
    /// </summary>
    public SldWorks swApp;
}
}
```

Select Multiple Splines for Loft Guide Curves Example (VBA)

This example shows how to select multiple splines for the guide curves for a loft feature.

```
'-----
' Preconditions: Verify that the specified part template exists.
'
' Postconditions:
' 1. Creates a new part.
' 2. Creates a profile sketch.
' 3. Creates a reference plane and another profile sketch on that
'    reference plane.
' 4. Creates two splines for the guide curves.
' 5. Selects the profile sketches.
' 6. Selects the splines and groups them as an object.
' 7. Creates a loft feature.
' 8. Examine the FeatureManager design tree and graphics area.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSketchSegment As SldWorks.SketchSegment
Dim swSketchManager As SldWorks.SketchManager
Dim swRefPlane As SldWorks.RefPlane
Dim swFeatureManager As SldWorks.FeatureManager
Dim status As Boolean

Sub main()
    Set swApp = Application.SldWorks

    'Create a new part
    Set swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)

    'Create a profile sketch
    Set swModelDocExt = swModel.Extension
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    swModel.ClearSelection2 True
    Set swSketchManager = swModel.SketchManager
    Set swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 7.06113079019074E-02, 0, 0, 0, 3.74944141689373E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Create a reference plane and another profile sketch
    'on that reference plane
    status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
    Set swFeatureManager = swModel.FeatureManager
    Set swRefPlane = swFeatureManager.InsertRefPlane(8, 0.07, 0, 0, 0, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Set swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 5.27205722070845E-02, 0, 0, 0, 1.54164850136235E-02, 0)
    swModel.ClearSelection2 True
    swSketchManager.InsertSketch True

    'Create a spline
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    Dim pointArray As Variant
    Dim points() As Double
    ReDim points(0 To 14) As Double
    points(0) = -0.07
    points(1) = 1.54164850136235E-02
    points(2) = 0
    points(3) = -5.31092941649547E-02
    points(4) = 2.80386111480766E-02
    points(5) = 0
    points(6) = -2.96934467839947E-02
    points(7) = 2.29795168190776E-02
    points(8) = 0
    points(9) = -1.12921067380967E-02
    points(10) = 0.026354325474415
    points(11) = 0
    points(12) = 0
    points(13) = 3.74944141689373E-02
    points(14) = 0
    pointArray = points
    Set swSketchSegment = swSketchManager.CreateSpline((pointArray))
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True

    'Create another spline
    status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
    ReDim points(0 To 8) As Double
    points(0) = -0.07
    points(1) = -1.54164850136235E-02
    points(2) = 0
    points(3) = -3.07689275649068E-02
    points(4) = -2.33694015292372E-02
    points(5) = 0
    points(6) = 0
    points(7) = -3.74944141689373E-02
    points(8) = 0
    pointArray = points
    Set swSketchSegment = swSketchManager.CreateSpline((pointArray))
    swSketchManager.InsertSketch True
    swModel.ClearSelection2 True
```

```
'Select the profile sketches
status = swModelDocExt.SelectByID("Sketch1", "SKETCH", -5.85496337278505E-02, 2.09585732143712E-02, 1, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID("Sketch2", "SKETCH", -3.79093739088495E-02, 1.07136192740755E-02, 1, True, 0, Nothing, 0)

'Select the splines for the guide curves
status = swModelDocExt.SelectByID("Spline1@Sketch3", "EXTSKETCHSEGMENT", -6.20659823337474E-03, 3.04187689522769E-02, 2, True, 0, Nothing, 0)
status = swModelDocExt.SelectByID("Spline1@Sketch4", "EXTSKETCHSEGMENT", -4.02947949143199E-02, -2.06106896601265E-02, 2, True, 0, Nothing, 0)
'Group the selected splines as an object
status = swModelDocExt.SelectByID("Unknown", "SELOBJGROUP", 0, 0, 0, True, 2, Nothing, 0)

'Create a loft
swFeatureManager.InsertProtrusionBlend2 False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True, 0
```

End Sub

Select Multiple Splines for Loft Guide Curves Example (VB.NET)

This example shows how to select multiple splines for the guide curves for a loft feature.

Preconditions: Verify that the specified part template exists.

Postconditions:

1. Creates a new part.
2. Creates a profile sketch.
3. Creates a reference plane and another profile sketch on that reference plane.
4. Creates two splines for the guide curves.
5. Selects the profile sketches.
6. Selects the splines and groups them as an object.
7. Creates a loft feature.
8. Examine the FeatureManager design tree and graphics area.

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```
        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim swSketchSegment As SketchSegment
        Dim swSketchManager As SketchManager
        Dim swRefPlane As RefPlane
        Dim swFeatureManager As FeatureManager
        Dim status As Boolean
```

```
        'Create a new part
```

```
        swModel = swApp.NewDocument("C:\ProgramData\SolidWorks\SOLIDWORKS 2017\templates\Part.prtdot", 0, 0, 0)
```

```
        'Create a profile sketch
```

```
        swModelDocExt = swModel.Extension
```

```
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        swModel.ClearSelection2(True)
```

```
        swSketchManager = swModel.SketchManager
```

```
        swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 0.0706113079019074, 0, 0, 0, 0.0374944141689373, 0)
```

```
        swModel.ClearSelection2(True)
```

```
        swSketchManager.InsertSketch(True)
```

```
        'Create a reference plane and another profile sketch
```

```
        'on that reference plane
```

```
        status = swModelDocExt.SelectByID("Front Plane", "PLANE", 0, 0, 0, True, 0, Nothing, 0)
```

```
        swFeatureManager = swModel.FeatureManager
```

```
        swRefPlane = swFeatureManager.InsertRefPlane(0, 0.07, 0, 0, 0, 0)
```

```
        swModel.ClearSelection2(True)
```

```
        status = swModelDocExt.SelectByID("Plane1", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        swSketchSegment = swSketchManager.CreateEllipse(0, 0, 0, 0.0527205722070845, 0, 0, 0, 0.0154164850136235, 0)
```

```
        swModel.ClearSelection2(True)
```

```
        swSketchManager.InsertSketch(True)
```

```
        'Create a spline
```

```
        status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        Dim pointArray As Object
```

```
        Dim points(14) As Double
```

```
        points(0) = 0.07
```

```
        points(1) = 0.0154164850136235
```

```
        points(2) = 0
```

```
        points(3) = -0.0531092941649547
```

```
        points(4) = 0.0280386111480766
```

```
        points(5) = 0
```

```
        points(6) = -0.0296934467839947
```

```
        points(7) = 0.0229795168190776
```

```
        points(8) = 0
```

```
        points(9) = -0.0112921067380967
```

```
        points(10) = 0.026354325474415
```

```
        points(11) = 0
```

```
        points(12) = 0
```

```
        points(13) = 0.0374944141689373
```

```
        points(14) = 0
```

```
        pointArray = points
```

```
        swSketchSegment = swSketchManager.CreateSpline((pointArray))
```

```
        swSketchManager.InsertSketch(True)
```

```
        swModel.ClearSelection2(True)
```

```
        'Create another spline
```

```
        status = swModelDocExt.SelectByID("Right Plane", "PLANE", 0, 0, 0, False, 0, Nothing, 0)
```

```
        ReDim points(8)
```

```
        points(0) = -0.07
```

```
        points(1) = -0.0154164850136235
```

```
        points(2) = 0
```

```
        points(3) = -0.0307689275649068
```

```
        points(4) = -0.0233694015292372
```

```
        points(5) = 0
```

```
        points(6) = 0
```

```
        points(7) = -0.0374944141689373
```

```
        points(8) = 0
```

```
        pointArray = points
```

```
        swSketchSegment = swSketchManager.CreateSpline((pointArray))
```

```
        swSketchManager.InsertSketch(True)
```

```
        swModel.ClearSelection2(True)
```

```
        'Select the profile sketches
```

```
        status = swModelDocExt.SelectByID("Sketch1", "SKETCH", -0.0585496337278505, 0.0209585732143712, 1, True, 0, Nothing, 0)
```

```
        status = swModelDocExt.SelectByID("Sketch2", "SKETCH", -0.0379093739088495, 0.0107136192740755, 1, True, 0, Nothing, 0)
```

```
        'Select the splines for the guide curves
```

```
        status = swModelDocExt.SelectByID("Spline1@Sketch3", "EXTSKETCHSEGMENT", -0.00620659823337474, 0.0304187689522769, 2, True, 0, Nothing, 0)
```

```
        status = swModelDocExt.SelectByID("Spline1@Sketch4", "EXTSKETCHSEGMENT", -0.0402947949143199, -0.0206106896601265, 2, True, 0, Nothing, 0)
```

```
        'Group the selected splines as an object
```

```
        status = swModelDocExt.SelectByID("Unknown", "SELOBJGROUP", 0, 0, 0, True, 2, Nothing, 0)
```

```
        'Create a loft
```

```
swFeatureManager.InsertProtrusionBlend2(False, True, False, 1, 0, 0, 1, 1, True, True, False, 0, 0, 0, True, True, True, 0)
```

```
End Sub
```

```
''' <summary>
```

```
''' The SldWorks swApp variable is pre-assigned for you.
```

```
''' </summary>
```

```
Public swApp As SldWorks
```

```
End Class
```

Select Near and Far Side Hole Wizard Countersink Features Example (C#)

This example shows how to select near and far side Hole Wizard countersink features.

```
//-----
// Preconditions:
// 1. Open a part that contains a Hole Wizard hole named, Hole1.
// 2. Modify the countersink diameter and angle values as required by Hole1.
// 3. Open an Immediate window.
//
// Postconditions:
// 1. Inspect the Immediate window.
// 2. The current near side countersink option is displayed.
// 3. The current far side countersink option is displayed.
// 4. The near side countersink option is toggled.
// 5. If the near side countersink option is selected,
//    the proper angle and diameter are set.
// 6. The far side countersink option is toggled.
// 7. If the far side countersink option is selected,
//    the proper angle and diameter are set.
// 8. The new near side countersink option is displayed.
// 9. The new far side countersink option is displayed.
//-----
using Microsoft.VisualBasic;
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data;
using System.Diagnostics;
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
namespace NearFarCounterSink_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        WizardHoleFeatureData2 FeatureData;
        Feature Feature;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelMgr;

        bool boolstatus;

        public void Main()
        {
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExt = swModel.Extension;
            swSelMgr = (SelectionMgr)swModel.SelectionManager;

            swModel.ClearSelection2(true);

            boolstatus = swModelDocExt.SelectByID("Hole1", "BODYFEATURE", 0, 0, 0, false, 0, null,
(int)swSelectOption_e.swSelectOptionDefault);
            Feature = (Feature)swSelMgr.GetSelectedObject6(1, -1);
            FeatureData = (WizardHoleFeatureData2)Feature.GetDefinition();

            Debug.Print("Near side countersink option is selected: " + FeatureData.NearSideCounterSink);

            if ((FeatureData.NearSideCounterSink))
            {
                FeatureData.NearSideCounterSink = false;
            }
            else
            {
                FeatureData.NearSideCounterSink = true;
                // near side countersink option is selected
                FeatureData.NearCounterSinkDiameter = 0.0061;
                // meters
                FeatureData.NearCounterSinkAngle = 1.57;
                // radians
            }

            Debug.Print("Far side countersink option is selected: " + FeatureData.FarSideCounterSink);

            if ((FeatureData.FarSideCounterSink))
            {
                FeatureData.FarSideCounterSink = false;
            }
            else
            {
                FeatureData.FarSideCounterSink = true;
                // far side countersink option is selected
                FeatureData.FarCounterSinkDiameter = 0.0061;
                // meters
                FeatureData.FarCounterSinkAngle = 1.57;
                // radians
            }
        }
    }
}
```

```
boolstatus = Feature.ModifyDefinition(FeatureData, swModel, null);
FeatureData = (WizardHoleFeatureData2)Feature.GetDefinition();

Debug.Print("Near side countersink option is selected: " + FeatureData.NearSideCounterSink);
Debug.Print("Far side countersink option is selected: " + FeatureData.FarSideCounterSink);

}

public SldWorks swApp;

}
```

Select Near and Far Side Hole Wizard Countersink Features Example (VBA)

This example shows how to select near and far side Hole Wizard countersink features.

```

'-----
' Preconditions:
' 1. Open a part that contains a Hole Wizard hole named, Hole1.
' 2. Modify the countersink diameter and angle values as required by Hole1.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The current near side countersink option is displayed.
' 3. The current far side countersink option is displayed.
' 4. The near side countersink option is toggled.
'   If the near side countersink option is selected,
'   the proper angle and diameter are set.
' 5. The far side countersink option is toggled.
'   If the far side countersink option is selected,
'   the proper angle and diameter are set.
' 6. The new near side countersink option is displayed.
' 7. The new far side countersink option is displayed.
'-----

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim FeatureData As SldWorks.WizardHoleFeatureData2
Dim Feature As SldWorks.Feature
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim boolstatus As Boolean
Option Explicit
Sub main()

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swModelDocExt = swModel.Extension
    Set swSelMgr = swModel.SelectionManager

    swModel.ClearSelection2 True

    boolstatus = swModelDocExt.SelectByID("Hole1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set Feature = swSelMgr.GetSelectedObject6(1, -1)
    Set FeatureData = Feature.GetDefinition

    Debug.Print "Near side countersink option is selected: " & FeatureData.NearSideCounterSink

    If (FeatureData.NearSideCounterSink) Then
        FeatureData.NearSideCounterSink = False
    Else
        FeatureData.NearSideCounterSink = True ' near side countersink option is selected
        FeatureData.NearCounterSinkDiameter = 0.0061 ' meters
        FeatureData.NearCounterSinkAngle = 1.57 ' radians
    End If

    Debug.Print "Far side countersink option is selected: " & FeatureData.FarSideCounterSink

    If (FeatureData.FarSideCounterSink) Then
        FeatureData.FarSideCounterSink = False
    Else
        FeatureData.FarSideCounterSink = True ' far side countersink option is selected
        FeatureData.FarCounterSinkDiameter = 0.0061 ' meters
        FeatureData.FarCounterSinkAngle = 1.57 ' radians
    End If

    boolstatus = Feature.ModifyDefinition(FeatureData, swModel, Nothing)
    Set FeatureData = Feature.GetDefinition

    Debug.Print "Near side countersink option is selected: " & FeatureData.NearSideCounterSink
    Debug.Print "Far side countersink option is selected: " & FeatureData.FarSideCounterSink

End Sub

```

Select Near and Far Side Hole Wizard Countersink Features Example (VB.NET)

This example shows how to select near and far side Hole Wizard countersink features.

```

'-----
' Preconditions:
' 1. Open a part that contains a Hole Wizard hole named, Hole1.
' 2. Modify the countersink diameter and angle values as required by Hole1.
' 3. Open an Immediate window.
'
' Postconditions:
' 1. Inspect the Immediate window.
' 2. The current near side countersink option is displayed.
' 3. The current far side countersink option is displayed.
' 4. The near side countersink option is toggled.
'    If the near side countersink option is selected,
'    the proper angle and diameter are set.
' 5. The far side countersink option is toggled.
'    If the far side countersink option is selected,
'    the proper angle and diameter are set.
' 6. The new near side countersink option is displayed.
' 7. The new far side countersink option is displayed.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim FeatureData As WizardHoleFeatureData2
    Dim Feature As Feature
    Dim swModelDocExt As ModelDocExtension
    Dim swSelMgr As SelectionMgr
    Dim boolstatus As Boolean

    Sub main()

        swModel = swApp.ActiveDoc
        swModelDocExt = swModel.Extension
        swSelMgr = swModel.SelectionManager

        swModel.ClearSelection2(True)

        boolstatus = swModelDocExt.SelectByID2("Hole1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing,
        swSelectOption_e.swSelectOptionDefault)
        Feature = swSelMgr.GetSelectedObject6(1, -1)
        FeatureData = Feature.GetDefinition

        Debug.Print("Near side countersink option is selected: " & FeatureData.NearSideCounterSink)

        If (FeatureData.NearSideCounterSink) Then
            FeatureData.NearSideCounterSink = False
        Else
            FeatureData.NearSideCounterSink = True ' near side countersink option is selected
            FeatureData.NearCounterSinkDiameter = 0.0061 ' meters
            FeatureData.NearCounterSinkAngle = 1.57 ' radians
        End If

        Debug.Print("Far side countersink option is selected: " & FeatureData.FarSideCounterSink)

        If (FeatureData.FarSideCounterSink) Then
            FeatureData.FarSideCounterSink = False
        Else
            FeatureData.FarSideCounterSink = True ' far side countersink option is selected
            FeatureData.FarCounterSinkDiameter = 0.0061 ' meters
            FeatureData.FarCounterSinkAngle = 1.57 ' radians
        End If

        boolstatus = Feature.ModifyDefinition(FeatureData, swModel, Nothing)
        FeatureData = Feature.GetDefinition

        Debug.Print("Near side countersink option is selected: " & FeatureData.NearSideCounterSink)
        Debug.Print("Far side countersink option is selected: " & FeatureData.FarSideCounterSink)

    End Sub

    Public swApp As SldWorks

End Class

```

Select Origin of Assembly Component Example (VBA)

This example shows how to get the origin of an assembly component.

```
'-----  
' Preconditions:  
' 1. Open an assembly that is fully resolved.  
' 2. Select a component.  
'  
' Postconditions:  
' 1. Selects the origin of the selected component.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

Sub main()

```
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swSelMgr As SldWorks.SelectionMgr  
Dim swSelComp As SldWorks.Component2  
Dim swCompModel As SldWorks.ModelDoc2  
Dim swFeat As SldWorks.Feature  
Dim bRet As Boolean
```

```
Set swApp = Application.SldWorks  
Set swModel = swApp.ActiveDoc  
Set swSelMgr = swModel.SelectionManager  
Set swSelComp = swSelMgr.GetSelectedObjectsComponent(1)  
Set swCompModel = swSelComp.GetModelDoc
```

```
swModel.ClearSelection2 True
```

```
Set swFeat = swSelComp.FirstFeature  
Do While Not swFeat Is Nothing  
If "OriginProfileFeature" = swFeat.GetTypeName Then  
bRet = swFeat.Select2(False, 0)  
Exit Do  
End If
```

```
Set swFeat = swFeat.GetNextFeature  
Loop  
End Sub
```

Select Plane Example (C#)

This example shows how to select the default Right Plane.

```
//-----
// Preconditions: Open a new part document.
//
// Postconditions:
// 1. Selects the Right Plane.
// 2. Examine the graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
namespace GetPlanes_CSharp.csproj
{
    partial class SolidWorksMacro
    {

        public void Main()
        {
            // Select Right Plane (1-based index)
            const long ReqPlane = 3;

            ModelDoc2 swModel = default(ModelDoc2);
            Feature swFeat = default(Feature);
            long PlaneCount = 0;
            bool bRet = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swFeat = (Feature)swModel.FirstFeature();

            while ((swFeat != null))
            {
                if ("RefPlane" == swFeat.GetTypeName())
                {
                    PlaneCount = PlaneCount + 1;

                    if (ReqPlane == PlaneCount)
                    {
                        bRet = swFeat.Select2(false, 0);
                        break;
                    }
                }

                swFeat = (Feature)swFeat.GetNextFeature();
            }

            public SldWorks swApp;
        }
    }
}
```

Select Plane Example (VBA)

This example shows how to select the default Right Plane.

```
'-----  
' Preconditions: Open a new part document.  
'  
' Postconditions:  
' 1. Selects the Right Plane.  
' 2. Examine the graphics area.  
'-----
```

Option Explicit

Sub main()

```
    ' Select Right Plane (1-based index)  
    Const ReqPlane          As Long = 3  
  
    Dim swApp                As SldWorks.SldWorks  
    Dim swModel              As SldWorks.ModelDoc2  
    Dim swFeat               As SldWorks.Feature  
    Dim PlaneCount           As Long  
    Dim bRet                 As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swFeat = swModel.FirstFeature  
  
    Do While Not swFeat Is Nothing  
        If "RefPlane" = swFeat.GetTypeFullName Then  
            PlaneCount = PlaneCount + 1  
  
            If ReqPlane = PlaneCount Then  
                bRet = swFeat.Select2(False, 0)  
                Exit Do  
            End If  
        End If  
  
        Set swFeat = swFeat.GetNextFeature  
    Loop  
End Sub
```

Select Plane Example (VB.NET)

This example shows how to select the default Right Plane.

```
'-----  
' Preconditions: Open a new part document.  
'
```

```
' Postconditions:  
' 1. Selects the Right Plane.  
' 2. Examine the graphics area.  
'-----
```

```
Imports SolidWorks.Interop.sldworks  
Imports SolidWorks.Interop.swconst  
Imports System.Runtime.InteropServices  
Imports System
```

```
Partial Class SolidWorksMacro
```

```
    Sub main()
```

```
        ' Select Right Plane (1-based index)  
        Const ReqPlane As Long = 3
```

```
        Dim swModel As ModelDoc2  
        Dim swFeat As Feature  
        Dim PlaneCount As Long  
        Dim bRet As Boolean
```

```
        swModel = swApp.ActiveDoc  
        swFeat = swModel.FirstFeature
```

```
        Do While Not swFeat Is Nothing  
            If "RefPlane" = swFeat.GetTypeName Then  
                PlaneCount = PlaneCount + 1  
  
                If ReqPlane = PlaneCount Then  
                    bRet = swFeat.Select2(False, 0)  
                    Exit Do  
                End If  
            End If
```

```
            swFeat = swFeat.GetNextFeature  
        Loop
```

```
    End Sub
```

```
    Public swApp As SldWorks
```

```
End Class
```

Select Silhouette Edge Attached to Note Example (C#)

This example shows how to select a silhouette edge attached to a note in a drawing.

```
//-----
// Preconditions:
// 1. Verify that the drawing to open exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified drawing.
// 2. Activates Drawing View1.
// 3. Gets the note in Drawing View1
//    and the silhouette edge to which
//    the note is attached.
// 4. Examine the Immediate window and graphics area.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Select2SilhouetteEdgeCSharp.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        DrawingDoc swDrawing;
        View swView;
        SilhouetteEdge swSilhouetteEdge;
        Note swNote;
        Annotation swAnnotation;
        bool status;
        int errors;
        int warnings;
        string fileName;
        object[] attachedEntities;

        public void Main()
        {
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2025\\samples\\tutorial\\api\\cylinder20.SLDDRW";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocDRAWING, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref errors, ref warnings);
            swDrawing = (DrawingDoc)swModel;
            status = swDrawing.ActivateView("Drawing View1");
            swView = (View)swDrawing.ActiveDrawingView;

            // Get note and any attached entities
            swNote = (Note)swView.GetFirstNote2();
            swAnnotation = (Annotation)swNote.GetAnnotation();
            attachedEntities = (object[])swAnnotation.GetAttachedEntities3();
            // Select the silhouette edge to which the note is attached
            foreach (object attachedEntity in attachedEntities)
            {
                swSilhouetteEdge = (SilhouetteEdge)attachedEntity;
                status = swSilhouetteEdge.Select2(false, null);
                if (status)
                {
                    Debug.Print("Silhouette edge selected.");
                }
            }
        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>

        public SldWorks swApp;
    }
}
```

Select Silhouette Edge Attached to Note Example (VBA)

This example shows how to select a silhouette edge attached to a note in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Activates Drawing View1.
' 3. Gets the note in Drawing View1
'    and the silhouette edge to which
'    the note is attached.
' 4. Examine the Immediate window and graphics area.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDrawing As SldWorks.DrawingDoc
Dim swView As SldWorks.View
Dim swSilhouetteEdge As SldWorks.SilhouetteEdge
Dim swNote As SldWorks.Note
Dim swAnnotation As SldWorks.Annotation
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim params As Variant
Dim obj As Variant

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\api\cylinder20.SLDDRW"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swDrawing = swModel
    status = swDrawing.ActivateView("Drawing View1")
    Set swView = swDrawing.ActiveDrawingView

    ' Get note and any attached entities
    Set swNote = swView.GetFirstNote2
    Set swAnnotation = swNote.GetAnnotation
    params = swAnnotation.GetAttachedEntities3
    ' Select the silhouette edge to which the note is attached
    For Each obj In params
        Set swSilhouetteEdge = obj
        status = swSilhouetteEdge.Select2(False, Nothing)
        If status Then
            Debug.Print ("Silhouette edge selected.")
        End If
    Next
End Sub
```

Select_Silhouette_Edge_Attached_to_Note_Example_VBNET (Topic)

SOLIDWORKS API Help

Select Silhouette Edge Attached to Note Example (VB.NET)

This example shows how to select a silhouette edge attached to a note in a drawing.

```
'-----
' Preconditions:
' 1. Verify that the drawing to open exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified drawing.
' 2. Activates Drawing View1.
' 3. Gets the note in Drawing View1
'    and the silhouette edge to which
'    the note is attached.
' 4. Examine the Immediate window and graphics area.
'-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Dim swModel As ModelDoc2
    Dim swDrawing As DrawingDoc
    Dim swView As View
    Dim swSilhouetteEdge As SilhouetteEdge
    Dim swNote As Note
    Dim swAnnotation As Annotation
    Dim status As Boolean
    Dim errors As Integer
    Dim warnings As Integer
    Dim fileName As String
    Dim params As Object
    Dim obj As Object

    Sub Main()

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2025\samples\tutorial\api\cylinder20.SLDDRW"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocDRAWING, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swDrawing = swModel
        status = swDrawing.ActivateView("Drawing View1")
        swView = swDrawing.ActiveDrawingView

        ' Get note and any attached entities
        swNote = swView.GetFirstNote2
        swAnnotation = swNote.GetAnnotation
        params = swAnnotation.GetAttachedEntities3
        ' Select the silhouette edge to which the note is attached
        For Each obj In params
            swSilhouetteEdge = obj
            status = swSilhouetteEdge.Select2(False, Nothing)
            If status Then
                Debug.Print("Silhouette edge selected.")
            End If
        Next

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class
```

Select_Table_Cells_Example_CSharp (Topic)

SOLIDWORKS API Help

Select Table Cells Example (C#)

This example shows how to interpret the results of selecting table cells in various ways.

```
//-----  
// Preconditions:  
// 1. A drawing with a table, which has multiple rows  
//    and columns, is open.  
// 2. Open the Immediate window.  
// 3. Select a column.  
// 4. Run the macro.  
//  
// Postconditions:  
// 1. The range for all of the cells in the column is  
//    printed to the Immediate window. Examine  
//    the Immediate window.  
// 2. Hold down the Ctrl key and click a cell.  
// 3. Repeat step 2 to select additional adjacent cells.  
// 4. Run the macro again.  
// 5. The range for each selected cell is printed to  
//    the Immediate window. Examine the Immediate  
//    window.  
//-----  
using SolidWorks.Interop.sldworks;  
using SolidWorks.Interop.swconst;  
using System.Runtime.InteropServices;  
using System;  
using System.Diagnostics;  
  
namespace GetCellRangeTableAnnotationCSharp.csproj  
{  
  
    partial class SolidWorksMacro  
    {  
        public void Main()  
        {  
            ModelDoc2 swModel = default(ModelDoc2);  
            SelectionMgr swSelectionMgr = default(SelectionMgr);  
            DrawingDoc swDrawing = default(DrawingDoc);  
            Annotation swAnnotation = default(Annotation);  
            TableAnnotation swTableAnnotation = default(TableAnnotation);  
            int firstRow = 0;  
            int lastRow = 0;  
            int firstColumn = 0;  
            int lastColumn = 0;  
            int idx = 0;  
  
            swModel = (ModelDoc2)swApp.ActiveDoc;  
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;  
            swDrawing = (DrawingDoc)swModel;  
  
            for (idx = 1; idx <= swSelectionMgr.GetSelectedObjectCount2(-1); idx++)  
            {  
                swTableAnnotation = (TableAnnotation)swSelectionMgr.GetSelectedObject6(idx, -1);  
                swAnnotation = (Annotation)swTableAnnotation.GetAnnotation();  
  
                swTableAnnotation.GetCellRange(ref firstRow, ref lastRow, ref firstColumn, ref lastColumn);  
  
                Debug.Print("First selected cell's row    = " + firstRow);  
                Debug.Print("Last selected cell's row    = " + lastRow);  
                Debug.Print("First selected cell's column = " + firstColumn);  
                Debug.Print("Last selected cell's column = " + lastColumn);  
                Debug.Print("");  
            }  
  
            if ((firstRow == -1))  
            {  
                Debug.Print("Selected entire table!");  
            }  
  
            swModel.ClearSelection2(true);  
        }  
    }  
  
    /// <summary>  
    /// The SldWorks swApp variable is pre-assigned for you.  
    /// </summary>  
  
    public SldWorks swApp;  
}
```

Select Table Cells Example (VBA)

This example shows how to interpret the results of selecting table cells in various ways.

```
'-----
' Preconditions:
' 1. A drawing with a table, which has multiple rows
'    and columns, is open.
' 2. Open the Immediate window.
' 3. Select a column.
' 4. Run the macro.
'
' Postconditions:
' 1. The range for all of the cells in the column is
'    printed to the Immediate window. Examine
'    the Immediate window.
' 2. Hold down the Ctrl key and click a cell.
' 3. Repeat step 2 to select additional adjacent cells.
' 4. Run the macro again.
' 5. The range for each selected cell is printed to
'    the Immediate window. Examine the Immediate
'    window.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
```

```
Sub main()
```

```
    Dim swModel          As SldWorks.ModelDoc2
    Dim swSelectionMgr    As SldWorks.SelectionMgr
    Dim swDrawing         As SldWorks.DrawingDoc
    Dim swAnnotation      As SldWorks.Annotation
    Dim swTableAnnotation As SldWorks.TableAnnotation
    Dim firstRow          As Long
    Dim lastRow           As Long
    Dim firstColumn       As Long
    Dim lastColumn        As Long
    Dim idx               As Long
```

```
    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swSelectionMgr = swModel.SelectionManager
    Set swDrawing = swModel
```

```
    For idx = 1 To swSelectionMgr.GetSelectedObjectCount2(-1)
```

```
        Set swTableAnnotation = swSelectionMgr.GetSelectedObject6(idx, -1)
        Set swAnnotation = swTableAnnotation.GetAnnotation
```

```
        swTableAnnotation.GetCellRange firstRow, lastRow, firstColumn, lastColumn
```

```
        Debug.Print "First selected cell's row    = " & firstRow
        Debug.Print "Last selected cell's row     = " & lastRow
        Debug.Print "First selected cell's column = " & firstColumn
        Debug.Print "Last selected cell's column  = " & lastColumn
        Debug.Print ""
```

```
    Next idx
```

```
    If (firstRow = -1) Then
        Debug.Print "Selected entire table!"
    End If
```

```
    swModel.ClearSelection2 True
```

```
End Sub
```

Select Table Cells Example (VB.NET)

This example shows how to interpret the results of selecting table cells in various ways.

```
.....
' Preconditions:
' 1. A drawing with a table, which has multiple rows
'    and columns, is open.
' 2. Open the Immediate window.
' 3. Select a column.
' 4. Run the macro.
'
' Postconditions:
' 1. The range for all of the cells in the column is
'    printed to the Immediate window. Examine
'    the Immediate window.
' 2. Hold down the Ctrl key and click a cell.
' 3. Repeat step 2 to select additional adjacent cells.
' 4. Run the macro again.
' 5. The range for each selected cell is printed to
'    the Immediate window. Examine the Immediate
'    window.
.....
```

```
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics
```

Partial Class SolidWorksMacro

```
Public Sub Main()

    Dim swModel As ModelDoc2
    Dim swSelectionMgr As SelectionMgr
    Dim swDrawing As DrawingDoc
    Dim swAnnotation As Annotation
    Dim swTableAnnotation As TableAnnotation
    Dim firstRow As Integer
    Dim lastRow As Integer
    Dim firstColumn As Integer
    Dim lastColumn As Integer
    Dim idx As Integer

    swModel = swApp.ActiveDoc
    swSelectionMgr = swModel.SelectionManager
    swDrawing = swModel

    For idx = 1 To swSelectionMgr.GetSelectedObjectCount2(-1)

        swTableAnnotation = swSelectionMgr.GetSelectedObject6(idx, -1)
        swAnnotation = swTableAnnotation.GetAnnotation

        swTableAnnotation.GetCellRange(firstRow, lastRow, firstColumn, lastColumn)

        Debug.Print("First selected cell's row    = " & firstRow)
        Debug.Print("Last selected cell's row     = " & lastRow)
        Debug.Print("First selected cell's column = " & firstColumn)
        Debug.Print("Last selected cell's column  = " & lastColumn)
        Debug.Print("")

    Next idx

    If (firstRow = -1) Then
        Debug.Print("Selected entire table!")
    End If

    swModel.ClearSelection2(True)

End Sub

''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

End Class

Select Tangent Edges via Iteration Example (VBA)

This example shows how to select tangent edges by iterating through an edge's coedges and selecting all edges with tangent vectors.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing document.
' 2. Select an edge tangent to other edges in the model.
'
' Postconditions:
' 1. Selects all edges tangent to the selected edge.
' 2. Examine the graphics area.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swDoc As SldWorks.ModelDoc2
Dim swSelMgr As SldWorks.SelectionMgr
Dim swEdge As SldWorks.Edge
Dim swSelData As SldWorks.SelectData
```

Sub main()

```
    ' Access the running SOLIDWORKS Design application
    Set swApp = GetObject(, "SldWorks.Application")
    If swApp Is Nothing Then Exit Sub

    ' Get the active model
    Set swDoc = swApp.ActiveDoc
    If swDoc Is Nothing Then Exit Sub

    ' Get the SelectionMgr object and make sure that the selected object is an edge
    Set swSelMgr = swDoc.SelectionManager
    If swSelMgr.GetSelectedObjectCount <> 1 Then Exit Sub
    If swSelMgr.GetSelectedObjectType2(1) <> swSelEdges Then Exit Sub
    Set swSelData = swSelMgr.CreateSelectData

    ' Get a pointer to the selected edge
    Set swEdge = swSelMgr.GetSelectedObject6(1, -1)

    ' Select all edges that are tangent to the selected edge
    Call SelectTangentEdges(swEdge)
```

End Sub

```
Sub SelectTangentEdges(swEdge As Edge)
    Dim varCoEdges As Variant      ' Coedges for edge
    Dim swCoEdge As SldWorks.CoEdge ' Holds the current coedge
    Dim bUseStart As Boolean        ' Gets whether the coedge is in
                                    ' the same direction as edge
                                    ' Determines where to evaluate the edge from
    Dim varTangent As Variant      ' The tangent to edge

    ' Get all of the coedges for this edge
    varCoEdges = swEdge.GetCoEdges

    ' If this edge does not have exactly two coedges, then quit
    If UBound(varCoEdges) <> 1 Then Exit Sub

    ' For the first coedge
    Set swCoEdge = varCoEdges(0)

    ' Depends on whether the coedge and edge have the same direction
    bUseStart = swCoEdge.GetSense
    If bUseStart Then bUseStart = False Else bUseStart = True

    ' Get the tangent for this edge, bUseStart indicates from where to get tangent
    varTangent = GetEdgeTangent(swEdge, bUseStart)

    ' Recurse for all tangent edges for this edge starting with coedge
    Call TraverseTangentEdges(swEdge, swCoEdge, varTangent, swSelData)

    ' For the second coedge
    Set swCoEdge = varCoEdges(1)

    ' Depends on whether the coedge and edge have the same direction
    bUseStart = swCoEdge.GetSense
    If bUseStart Then bUseStart = False Else bUseStart = True

    ' Get the tangent for this edge, bUseStart indicates from where to get tangent
    varTangent = GetEdgeTangent(swEdge, bUseStart)

    ' Recurse for all tangent edges for this edge starting with coedge
    Call TraverseTangentEdges(swEdge, swCoEdge, varTangent, swSelData)
```

End Sub

```
' Selects all tangent edges starting at edge, and continuing out from coedge
Sub TraverseTangentEdges(swEdge As Edge, swCoEdge As CoEdge, varTangent As Variant, swSelData)

    Dim swNextCoEdge As SldWorks.CoEdge      ' Next coedge on coedge
    Dim swPartnerCoEdge As SldWorks.CoEdge   ' Partner to the coedge
    Dim swNextEdge As SldWorks.Edge          ' Next coedge
    Dim swEntity As SldWorks.Entity          ' Used to select the edge
    Dim varNextTangent As Variant            ' Holds the tangent for the next coedge
```

```

Dim bUseNextStart As Boolean          ' Get whether the next tangent comes from the
                                      ' start of the edge or not

' Get the next edge
Set swNextCoEdge = swCoEdge.GetNext
Set swNextEdge = swNextCoEdge.GetEdge

'While not visiting the edge started on
Do While Not swNextEdge Is swEdge
    bUseNextStart = swNextCoEdge.GetSense
    varNextTangent = GetEdgeTangent(swNextEdge, bUseNextStart)
    If VectorsAreEqual(varTangent, varNextTangent) Then
        Set swEntity = swNextEdge
        ' Select the edge and append it to the list of selected items
        swEntity.Select4 True, swSelData
        ' Go to the next
        If bUseNextStart Then bUseNextStart = False Else bUseNextStart = True
        varNextTangent = GetEdgeTangent(swNextEdge, bUseNextStart)
        Call TraverseTangentEdges(swNextEdge, swNextCoEdge, varNextTangent, swSelData)
        Exit Sub
    Else
        ' Walk around the vertex
        Set swPartnerCoEdge = swNextCoEdge.GetPartner
        Set swNextCoEdge = swPartnerCoEdge.GetNext
        Set swNextEdge = swNextCoEdge.GetEdge
    End If
Loop
End Sub

' Get the tangent for the current edge
Function GetEdgeTangent(swEdge As Edge, bUseStart As Boolean) As Variant
    Dim swCurve As SldWorks.Curve      ' swEdge's underlying curve
    Dim varEdgeParams As Variant        ' Curve parameters for swEdge
    Dim dblParam As Double              ' Edge parameter to evaluate
    Dim dblTangent(2) As Double         ' Tangent for swEdge

    ' Get the curve parameters for swEdge
    Set swCurve = swEdge.GetCurve
    varEdgeParams = swEdge.GetCurveParams2

    ' If the coedge has the same direction as swEdge, then evaluate from
    ' the start of the coedge; otherwise, evaluate from the end of
    ' the coedge (the start of the edge)
    If bUseStart Then
        dblParam = varEdgeParams(6)
    Else
        dblParam = varEdgeParams(7)
    End If

    ' Evaluate the edge at the start or end
    varEdgeParams = swEdge.Evaluate(dblParam)
    ' Get the tangent information out of the evaluation
    dblTangent(0) = varEdgeParams(3)
    dblTangent(1) = varEdgeParams(4)
    dblTangent(2) = varEdgeParams(5)

    'Return the tangent
    GetEdgeTangent = dblTangent
End Function

' Determines if the edges are equal within a tolerance of 1.0 * e^-10
Function VectorsAreEqual(varVec1 As Variant, varVec2 As Variant) As Boolean
    Dim dblMag As Double
    Dim dblDot As Double
    Dim dblUnit1(2) As Double
    Dim dblUnit2(2) As Double

    dblMag = (varVec1(0) * varVec1(0) + varVec1(1) * varVec1(1) + varVec1(2) * varVec1(2)) ^ 0.5
    dblUnit1(0) = varVec1(0) / dblMag: dblUnit1(1) = varVec1(1) / dblMag: dblUnit1(2) = varVec1(2) / dblMag
    dblMag = (varVec2(0) * varVec2(0) + varVec2(1) * varVec2(1) + varVec2(2) * varVec2(2)) ^ 0.5
    dblUnit2(0) = varVec2(0) / dblMag: dblUnit2(1) = varVec2(1) / dblMag: dblUnit2(2) = varVec2(2) / dblMag
    dblDot = dblUnit1(0) * dblUnit2(0) + dblUnit1(1) * dblUnit2(1) + dblUnit1(2) * dblUnit2(2)
    dblDot = Abs(Abs(dblDot) - 1#)

    If dblDot < 0.0000000001 Then '1.0e-10
        VectorsAreEqual = True
    Else
        VectorsAreEqual = False
    End If
End Function

```

Select Tangent Edges Topologically Example (VBA)

This example shows how to select tangent edges using `IEdges::GetTangentEdges`, which is a precise topological method.

NOTE: `IEdge::GetTangentEdges` does not give same results as `IModelDoc2::SelectTangency`, which behaves the same as if selecting tangent edges in the user interface. Because the user interface functionality automatically takes into account edges that lie on top of each other, `IModelDoc2::SelectTangency` is considered to be imprecise.

```
'-----  
' Preconditions: Open a part and select an edge  
' tangent to other edges.  
'  
' Postconditions:  
' 1. Selects the edges tangent to the selected edge.  
' 2. Examine the graphics area.  
'-----  
Option Explicit  
  
Sub main()  
  
    Dim swApp As SldWorks.SldWorks  
    Dim swModel As SldWorks.ModelDoc2  
    Dim swSelMgr As SldWorks.SelectionMgr  
    Dim swSelEdge As SldWorks.Edge  
    Dim vTanEdgeArr As Variant  
    Dim vTanEdge As Variant  
    Dim swTanEdge As SldWorks.Edge  
    Dim swTanEnt As SldWorks.Entity  
    Dim bRet As Boolean  
  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swSelMgr = swModel.SelectionManager  
    Set swSelEdge = swSelMgr.GetSelectedObject6(1, -1)  
  
    vTanEdgeArr = swSelEdge.GetTangentEdges: If IsEmpty(vTanEdgeArr) Then Exit Sub  
    For Each vTanEdge In vTanEdgeArr  
        Set swTanEdge = vTanEdge  
        Set swTanEnt = swTanEdge  
        bRet = swTanEnt.Select4(True, Nothing): Debug.Assert bRet  
    Next vTanEdge  
  
End Sub
```

Select Tangent Faces Example (VBA)

This example shows how to select all faces tangent to a selected face.

1. Starting with the selected face, get the first loop in the face with `IFace2::GetFirstLoop`.
2. Starting with the first loop and using `ILoop2::GetNext` to get every loop on the face:
 - a. Iterate through every coedge contained in the loop using `ILoop2::GetFirstCoEdge` and `ILoop2::GetNext`.
 - b. Using `ICoEdge::GetPartner`, get the partner coedge.
 - c. For the current coedge and the partner coedge, get the face normal at the middle of the coedge. This is done by calling `ICoEdge::Evaluate` to get the middle point of the coedge, then calling `ISurface::EvaluateAtPoint` at the middle point on the surface that contains the coedge, and finally extracting the normal vector from the return value of `ISurface::EvaluateAtPoint`.
 - d. If the normal vectors are equal, then use the partner coedge to get the partner loop, which provides access to the partner face that can be selected.

```
'-----
' Preconditions:
' 1. Open a part, assembly, or drawing document.
' 2. Select a face on the model.
'
' Postconditions:
' 1. Selects all faces tangent to the selected face.
' 2. Examine the graphics area.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swDoc As SldWorks.ModelDoc2
    Dim swSelMgr As SldWorks.SelectionMgr
    Dim swFace As SldWorks.Face2
    Dim swSelData As SldWorks.SelectData

    Set swApp = GetObject(, "SldWorks.Application")
    Set swDoc = swApp.ActiveDoc

    'Get the ISelectionMgr and make sure that a face is selected
    Set swSelMgr = swDoc.SelectionManager
    If swSelMgr.GetSelectedObjectCount <> 1 Then Exit Sub
    If swSelMgr.GetSelectedObjectType2(1) <> swSelFACES Then Exit Sub
    Set swSelData = swSelMgr.CreateSelectData

    'Select all faces that are tangent to the selected face
    Set swFace = swSelMgr.GetSelectedObject6(1, -1)

    Call SelectTangentFaces(swFace, swSelData)

End Sub

'This subroutine selects all faces that are tangent to the given face
Sub SelectTangentFaces(swFace As Object, swSelData As Object)
    Dim swPartnerFace As SldWorks.Face2      'Partner face for the current face
    Dim swThisLoop As SldWorks.Loop2         'Current loop
    Dim swPartnerLoop As SldWorks.Loop2      'Partner to the current loop
    Dim swFirstCoEdge As SldWorks.CoEdge     'First coedge on a loop
    Dim swThisCoEdge As SldWorks.CoEdge     'Current coedge
    Dim swPartnerCoEdge As SldWorks.CoEdge   'Partner to the current coedge
    Dim swEntity As Entity                   'Entity containing the face to be selected
    Dim varThisNormal As Variant             'Normal vector for the current face
    Dim varPartnerNormal As Variant          'Normal vector for the partner face

    Set swThisLoop = swFace.GetFirstLoop

    'For every loop on this face
    Do While Not swThisLoop Is Nothing
        'Get the first coedge of this loop
        Set swFirstCoEdge = swThisLoop.GetFirstCoEdge
        Set swThisCoEdge = swFirstCoEdge
        'Iterate through the coedges
        Do
            Set swPartnerCoEdge = swThisCoEdge.GetPartner
            'Get the normal vectors for the face, and its partner
            varThisNormal = GetFaceNormalAtMidCoEdge(swThisCoEdge)
            varPartnerNormal = GetFaceNormalAtMidCoEdge(swPartnerCoEdge)
            'If the normals are equal, then the faces are tangent
            'and the partner face should be selected
            If VectorsAreEqual(varThisNormal, varPartnerNormal) = True Then
                Set swPartnerLoop = swPartnerCoEdge.GetLoop
                Set swPartnerFace = swPartnerLoop.GetFace
                Set swEntity = swPartnerFace
                swEntity.Select4 True, swSelData
            End If

            ' Move to the next
            Set swThisCoEdge = swThisCoEdge.GetNext
            If swThisCoEdge Is swFirstCoEdge Then Exit Do
        Loop
        ' Move to the next loop
        Set swThisLoop = swThisLoop.GetNext
    Loop
End Sub

'This function returns the normal vector for the face at the provided coedge
Function GetFaceNormalAtMidCoEdge(swCoEdge As Object) As Variant
    Dim swFace As SldWorks.Face2            'Face containing swCoEdge
    Dim swSurface As SldWorks.Surface        'Surface containing swCoEdge,
```

```

                                'used to get the normal
Dim swLoop As SldWorks.Loop2      'Loop containing swCoEdge
Dim varParams As Variant          'Curve parameters for swCoEdge
Dim varPoint As Variant           'Array containing the location dblMidParam
Dim dblMidParam As Double         'Middle of the coedge
Dim dblNormal(2) As Double       'New vector

varParams = swCoEdge.GetCurveParams
'Set the evaluation point based on the coedge curve parameters
If varParams(6) > varParams(7) Then
    dblMidParam = (varParams(6) - varParams(7)) / 2 + varParams(7)
Else
    dblMidParam = (varParams(7) - varParams(6)) / 2 + varParams(6)
End If

'Get the location of the middle of the coedge
varPoint = swCoEdge.Evaluate(dblMidParam)

'Obtain the surface that contains swCoEdge
Set swLoop = swCoEdge.GetLoop
Set swFace = swLoop.GetFace
Set swSurface = swFace.GetSurface

'varParams contains the normal vector at the provided points (midpoint of edge)
varParams = swSurface.EvaluateAtPoint(varPoint(0), varPoint(1), varPoint(2))

'Build the normal vector
dblNormal(0) = varParams(0)
dblNormal(1) = varParams(1)
dblNormal(2) = varParams(2)

'Return the normal vector
GetFaceNormalAtMidCoEdge = dblNormal
End Function

'Determines whether two vectors are equal within a tolerance of 1.0 * e^-10
Function VectorsAreEqual(varVec1 As Variant, varVec2 As Variant) As Boolean
    Dim dblMag As Double
    Dim dblDot As Double
    Dim dblUnit1(2) As Double
    Dim dblUnit2(2) As Double

    dblMag = (varVec1(0) * varVec1(0) + varVec1(1) * varVec1(1) + varVec1(2) * varVec1(2)) ^ 0.5
    dblUnit1(0) = varVec1(0) / dblMag: dblUnit1(1) = varVec1(1) / dblMag: dblUnit1(2) = varVec1(2) / dblMag
    dblMag = (varVec2(0) * varVec2(0) + varVec2(1) * varVec2(1) + varVec2(2) * varVec2(2)) ^ 0.5
    dblUnit2(0) = varVec2(0) / dblMag: dblUnit2(1) = varVec2(1) / dblMag: dblUnit2(2) = varVec2(2) / dblMag
    dblDot = dblUnit1(0) * dblUnit2(0) + dblUnit1(1) * dblUnit2(1) + dblUnit1(2) * dblUnit2(2)
    dblDot = Abs(Abs(dblDot) - 1#)
    If dblDot < 0.000000001 Then '1.0e-10
        VectorsAreEqual = True
    Else
        VectorsAreEqual = False
    End If
End Function

```

Selection Lists Example (VBA)

If you specify a positive, non-0 mark for an object (e.g., 1 or 2 or 3, etc.) and then specify to select only the objects with that mark, then the resulting selection list contains only those objects with that mark. This selection list is independent of a selection list created by selecting all of the objects.

```
'-----
' Preconditions:
' 1. Verify that the specified part document to open exists.
' 2. Examine the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects all of the extrude features and marks Base-Extrude
'    and Boss-Extrude5 with a value of 8.
' 3. Gets the number of marked selections and total number of
'    selections.
' 4. Examine the Immediate window.
'-----
```

Option Explicit

```
Dim swApp                As SldWorks.SldWorks
Dim swModel              As SldWorks.ModelDoc2
Dim swDocSpecification    As SldWorks.DocumentSpecification
Dim swSelMgr             As SldWorks.SelectionMgr
Dim swModelExt           As SldWorks.ModelDocExtension
Dim bRet                 As Boolean
Dim lMark                As Long
Dim lMarkedIdx           As Long
Dim lNumMarkedSelections As Long
Dim lNumAllSelections    As Long
Dim lAllIdx              As Long
Dim swMarkedSelectedObject As Object
Dim swSelectedObject     As Object

Sub main()

    Set swApp = Application.SldWorks

    ' Open document
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\swutilities\bracket_a.sldprt")
    Set swModel = swApp.ActiveDoc
    If swModel Is Nothing Then
        Set swModel = swApp.OpenDoc7(swDocSpecification)
    End If
    Set swSelMgr = swModel.SelectionManager
    Set swModelExt = swModel.Extension

    ' Select all of the extrude features; Base-Extrude and Boss-Extrude5 are marked with a value of 8
    bRet = swModelExt.SelectByID2("Base-Extrude", "BODYFEATURE", 0, 0, 0, True, 8, Nothing, swSelectOptionDefault)
    bRet = swModelExt.SelectByID2("Boss-Extrude3", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, swSelectOptionDefault)
    bRet = swModelExt.SelectByID2("Boss-Extrude4", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, swSelectOptionDefault)
    bRet = swModelExt.SelectByID2("Boss-Extrude5", "BODYFEATURE", 0, 0, 0, True, 8, Nothing, swSelectOptionDefault)
    bRet = swModelExt.SelectByID2("Boss-Extrude7", "BODYFEATURE", 0, 0, 0, True, 0, Nothing, swSelectOptionDefault)

    ' Look for a given mark value
    lMark = 8

    ' Get the number of marked selections
    lNumMarkedSelections = swSelMgr.GetSelectedObjectCount2(lMark)
    Debug.Print "Number of marked selections = " & lNumMarkedSelections

    ' Get the total number of selections
    lNumAllSelections = swSelMgr.GetSelectedObjectCount2(-1)
    Debug.Print "Number of selections = " & lNumAllSelections
    Debug.Print " "

    For lMarkedIdx = 1 To lNumMarkedSelections
        Set swMarkedSelectedObject = swSelMgr.GetSelectedObject6(lMarkedIdx, lMark)
        For lAllIdx = 1 To lNumAllSelections
            Set swSelectedObject = swSelMgr.GetSelectedObject6(lAllIdx, -1)
            If (swMarkedSelectedObject Is swSelectedObject) Then
                ' Types must match
                Debug.Assert (swSelMgr.GetSelectedObjectType3(lMarkedIdx, lMark) = swSelMgr.GetSelectedObjectType3(lAllIdx, -1))
                Debug.Print "Number of marked selections = " & lMarkedIdx
                Debug.Print "Number of selections = " & lAllIdx
                Debug.Print " "
            End If
        Next lAllIdx
    Next lMarkedIdx

    swModel.ClearSelection2 True

End Sub
```

SelectionManager.cpp Written in C++

```
// SelectionManager.cpp : Defines the entry point for the console application
// Gets the selected objects in the active model document

#include "stdafx.h"

// Add the path to the SOLIDWORKS type libraries to the "Additional Include Directories".
#import "sldworks.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS type library

#import "swconst.tlb" raw_interfaces_only, raw_native_types, no_namespace, named_guids // SOLIDWORKS constants type library

int _tmain(int argc, _TCHAR* argv[])
{
    // Initialize COM
    // Do this before using ATL smart pointers so COM is available
    CoInitialize(NULL);

    // Use a block so the smart pointers are destructed when the scope of this block is left
    {
        // Use ATL smart pointers
        CComPtr<ISldWorks> swApp;

        if(swApp.CoCreateInstance(__uuidof(SldWorks), NULL, CLSCTX_LOCAL_SERVER) != S_OK) {
            return(0);
        }

        swApp->put_UserControl(VARIANT_TRUE);
        swApp->put_Visible(VARIANT_TRUE);

        CComPtr<IModelDoc2> swModel;

        swApp->get_IActiveDoc2(&swModel);

        if (! swModel) {
            return(0);
        }

        CComBSTR strModelTitle;
        long nDocumentType; // swDocumentTypes_e

        swModel->GetTitle(&strModelTitle);
        swModel->GetType(&nDocumentType);

        CComPtr<ISelectionMgr> swSelectionManager;
        long lNumSelections;
        long nSelectionType; // swSelectType_e
        CComPtr<IDispatch> swSelectedObject;
        CComPtr<IFace2> swFace;
        CComPtr<IEdge> swEdge;

        swModel->get_ISelectionManager(&swSelectionManager);

        swSelectionManager->GetSelectedObjectCount2(-1, &lNumSelections);

        for (int i = 1; i <= lNumSelections; i++) {

            swSelectionManager->GetSelectedObject6(i, -1, &swSelectedObject);

            swSelectionManager->GetSelectedObjectType3(i, -1, &nSelectionType);

            switch (nSelectionType) {

                case swSelectType_e::swSelFACES:
                    swSelectedObject.QueryInterface(&swFace);
                    break;
            }
        }
    }
}
```

```
        case swSelectType_e::swSeLEDGES:
            swSelectedObject.QueryInterface(&swEdge);
            break;
    }
}

// ATL smart pointers are destructed so all COM objects you held on to are released
// Now you can safely shut down COM as you do not need it any longer

// Stop COM
CoUninitialize();

return(0);
}
```

SelectionManager.cpp Written in C++/CLI

```
// SelectionManager.cpp : main project file
// Gets the selected objects in the active model document

#include "stdafx.h"

using namespace System;

// Add the SOLIDWORKS primary interop assemblies to the references
using namespace SOLIDWORKS::Interop::sldworks;
using namespace SOLIDWORKS::Interop::swconst;

int main(array<System::String ^> ^args)
{
    ISldWorks^    swApp;

    swApp = gcnew SldWorksClass;

    if (! swApp) {
        return(0);
    }

    swApp->UserControl = true;
    swApp->Visible = true;

    IModelDoc2^  swModel;

    swModel = swApp->IActiveDoc2;

    if (! swModel) {
        return(0);
    }

    String^      strModelTitle;
    swDocumentTypes_e  nDocumentType;

    strModelTitle = swModel->GetTitle();
    nDocumentType = (swDocumentTypes_e)swModel->GetType();

    ISelectionMgr^    swSelectionManager;
    swSelectType_e    nSelectionType;
    Object^           swSelectedObject;
    IFace2^           swFace;
    IEdge^            swEdge;

    swSelectionManager = swModel->ISelectionManager;

    for (int i = 1; i <= swSelectionManager->GetSelectedObjectCount2(-1); i++) {
        swSelectedObject = swSelectionManager->GetSelectedObject6(i, -1);
        nSelectionType = (swSelectType_e)swSelectionManager->GetSelectedObjectType3(i, -1);
        switch (nSelectionType) {
            case swSelectType_e::swSelFACES:
                swFace = (Face2^)swSelectedObject;
                break;
            case swSelectType_e::swSelEDGES:
                swEdge = (Edge^)swSelectedObject;
                break;
        }
    }

    return(0);
}
```

Selective Open Post-Notify Event Example (C#)

This example shows how to handle the post-notification event that fires when components are selected for Quick View/Selective Open.

```
//-----
// Preconditions:
// 1. Verify that the specified assembly to open
//    exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. If the Large Design Review dialog displays,
//    then click OK to close it.
// 2. When prompted, select the components to open
//    and click Open Selected.
// 3. Click OK to close the message box.
// 4. If prompted to rebuild, then click Rebuild.
// 5. After reading the next message box displayed,
//    click OK to close it.
// 6. Displays only the selected components.
// 7. Inspect the Immediate Window and graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not save
// changes.
//-----
using SolidWorks.Interop.sldworks;

using SolidWorks.Interop.swconst;

using System;

namespace SelectiveOpenPostNotify_CSharp.csproj
{
    partial class SolidWorksMacro
    {
        public AssemblyDoc swAssembly;
        public void Main()
        {
            swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, false);
            ModelDoc2 swModel = default(ModelDoc2);
            DocumentSpecification swDocSpecification = default(DocumentSpecification);
            swDocSpecification = (DocumentSpecification)swApp.GetOpenDocSpec("C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS
2018\\samples\\tutorial\\advdrawings\\bowl and chute.sldasm");
            swDocSpecification.InteractiveComponentSelection = true;
            swDocSpecification.DocumentType = (int)swDocumentTypes_e.swDocASSEMBLY;
            swModel = swApp.OpenDoc7(swDocSpecification);
            swModel = (ModelDoc2)swApp.ActiveDoc;
            swAssembly = (AssemblyDoc)swModel;
            // Set up event
            AttachEventHandlers();
        }

        public void AttachEventHandlers()
        {
            AttachSWEvents();
        }

        public void AttachSWEvents()
        {
            swAssembly.SelectiveOpenPostNotify += this.swAssembly_SelectiveOpenPostNotify;
        }

        public int swAssembly_SelectiveOpenPostNotify(string NewAddedDisplayStateName, ref object SelectedComponentNames)
        {
            System.Windows.Forms.MessageBox.Show("A post-notification event has been fired for the selective open.");
            System.Diagnostics.Debug.Print("New display state name: " + NewAddedDisplayStateName);
            System.Diagnostics.Debug.Print("Selected component names:");
            long i = 0;
            String[] selected;
            selected = (String[])SelectedComponentNames;
            for (i = 0; i <= selected.GetUpperBound(0); i++)
            {
                System.Diagnostics.Debug.Print(" " + selected[i]);
            }
        }
    }
}
```

```
        swApp.SetUserPreferenceToggle((int)swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, true);
        return 0;
    }
    public SldWorks swApp;
}
}
```

Selective Open Post-Notify Event Example (VBA)

This example shows how to handle the post-notification event that fires when components are selected for Quick View/Selective Open.

```
'-----
' Preconditions:
' 1. Copy Main into the macro.
' 2. Click Insert > Class Module and copy Class1
'    into that module.
' 3. Verify that the specified assembly to open
'    exists.
' 4. Open the Immediate window.
'
' Postconditions:
' 1. If the Large Design Review dialog displays,
'    then click OK to close it.
' 2. When prompted, select the components to open
'    and click Open Selected.
' 3. Click OK to close the message box.
' 4. If prompted to rebuild, then click Rebuild.
' 5. After reading the next message box displayed,
'    click OK to close it.
' 6. Displays only the selected components.
' 7. Inspect the Immediate Window and graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save
' changes.
'-----

'Main
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swDocSpecification As SldWorks.DocumentSpecification
Dim swAssemblyEvents As Class1
Dim swAssembly As SldWorks.AssemblyDoc
Option Explicit
Sub main()
    Set swApp = Application.SldWorks
    Set swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\ Bowl and chute.sldasm")
    swDocSpecification.InteractiveComponentSelection = True
    swDocSpecification.DocumentType = swDocASSEMBLY
    Set swModel = swApp.OpenDoc7(swDocSpecification)

    Set swModel = swApp.ActiveDoc
    Set swAssembly = swModel

    ' Set up event

    Set swAssemblyEvents = New Class1
    Set swAssemblyEvents.swAssembly = swAssembly
End Sub

'Class1

Public WithEvents swAssembly As SldWorks.AssemblyDoc
Private Function swAssembly_SelectiveOpenPostNotify(ByVal NewAddedDisplayStateName As String, SelectedComponentNames As Variant) As Long
    MsgBox "A post-notification event has been fired for the selective open."
    Debug.Print "New display state name: " & NewAddedDisplayStateName
    Debug.Print "Selected component names:"
    Dim i As Long
    For i = 0 To UBound(SelectedComponentNames)
        Debug.Print " " & SelectedComponentNames(i)
    Next
End Function
```

[Back to top](#)

[Back to top](#)

Selective Open Post-Notify Event Example (VB.NET)

This example shows how to handle the post-notification event that fires when components are selected for Quick View/Selective Open.

```
'-----
' Preconditions:
' 1. Verify that the specified assembly to open
'    exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. If the Large Design Review dialog displays,
'    then click OK to close it.
' 2. When prompted, select the components to open
'    and click Open Selected.
' 3. Click OK to close the message box.
' 4. If prompted to rebuild, then click Rebuild.
' 5. After reading the next message box displayed,
'    click OK to close it.
' 6. Displays only the selected components.
' 7. Inspect the Immediate Window and graphics area.
'
' NOTE: Because the assembly is used elsewhere, do not save
'       changes.
'-----

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System

Partial Class SolidWorksMacro
    Public WithEvents swAssembly As AssemblyDoc

    Public Sub main()
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, False)
        Dim swModel As ModelDoc2
        Dim swDocSpecification As DocumentSpecification
        swDocSpecification = swApp.GetOpenDocSpec("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS
2018\samples\tutorial\advdrawings\ Bowl and chute.sldasm")
        swDocSpecification.InteractiveComponentSelection = True
        swDocSpecification.DocumentType = swDocumentTypes_e.swDocASSEMBLY
        swModel = swApp.OpenDoc7(swDocSpecification)
        swModel = swApp.ActiveDoc
        swAssembly = swModel
        ' Set up event
        AttachEventHandlers()

    End Sub

    Sub AttachEventHandlers()
        AttachSWEvents()
    End Sub

    Sub AttachSWEvents()
        AddHandler swAssembly.SelectiveOpenPostNotify, AddressOf Me.swAssembly_SelectiveOpenPostNotify
    End Sub

    Public Function swAssembly_SelectiveOpenPostNotify(ByVal NewAddedDisplayStateName As String, ByRef SelectedComponentNames As
Object) As Integer
        MsgBox("A post-notification event has been fired for the selective open.")
        System.Diagnostics.Debug.Print("New display state name: " & NewAddedDisplayStateName)
        System.Diagnostics.Debug.Print("Selected component names:")
        Dim i As Long
        For i = 0 To UBound(SelectedComponentNames)
            System.Diagnostics.Debug.Print("  " & SelectedComponentNames(i))
        Next
        swApp.SetUserPreferenceToggle(swUserPreferenceToggle_e.swStopDebuggingVstaOnExit, True)
    End Function

    Public swApp As SldWorks
End Class
```

Selectively and Transparently Section a Section View Example (C#)

This example shows how to selectively and transparently section a section view.

```
//-----
// Preconditions: Verify that the assembly to open exists.
//
// Postconditions:
// 1. Opens the assembly.
// 2. Selects the component for selective sectioning.
// 3. Selects the components for transparent sectioning.
// 4. Selects the planes for sectioning.
// 5. Creates the SectionViewData object.
//    a. Sets the section method to zonal.
//    b. Sets to cap the sections.
//    c. Sets to generate a graphics-only section view.
//    d. Enables selective sectioning.
//        1. Sets the component selected in step 2 for selective
//           sectioning.
//        2. Sets the intersection zone.
//        3. Sets to exclude the component set in step 5.d.1 from
//           selective sectioning.
//    e. Enables transparent sectioning.
//        1. Sets the components selected in step 3
//           for transparent sectioning.
//        2. Sets to include the sectioned components set in step 5.e.1
//           in transparent sectioning.
//        3. Sets the level of transparency.
// 6. Creates the section view.
// 7. Examine the graphics area.
// 8. Click Section View twice in the Heads-Up View toolbar and
//    examine the Section View PropertyManager page.
//
// NOTE: Because the assembly is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {

        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            Component2[] selectiveArray = new Component2[1];
            Component2 selComponent = default(Component2);
            SelectionMgr swSelMgr = default(SelectionMgr);
            Component2[] transparentArray = new Component2[2];
            Component2 transComponent = default(Component2);
            Component2 transComponent2 = default(Component2);
            SectionViewData swSectionViewData = default(SectionViewData);
            ModelViewManager swModelViewManager = default(ModelViewManager);
            object transArray = new object[2];
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            object[] selArray = new object[1];

            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\floxpress\\ball valve\\ball_valve.sldasm";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocASSEMBLY, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref e
rrors, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;

            status = swModelDocExt.SelectByID2("Ball-1@ball_valve", "COMPONENT", 0, 0, 0, true, 8, null, 0);
            swSelMgr = (SelectionMgr)swModel.SelectionManager;
            selComponent = (Component2)swSelMgr.GetSelectedObject6(1, -1);
            selectiveArray[0] = selComponent;
            selArray = (object[])selectiveArray;

            status = swModelDocExt.SelectByID2("Side-1@ball_valve", "COMPONENT", 0, 0, 0, true, 32, null, 0);
            status = swModelDocExt.SelectByID2("Side-2@ball_valve", "COMPONENT", 0, 0, 0, true, 32, null, 0);
            transComponent = (Component2)swSelMgr.GetSelectedObject6(1, -1);
            transComponent2 = (Component2)swSelMgr.GetSelectedObject6(2, -1);
            transparentArray[0] = transComponent;
            transparentArray[1] = transComponent2;
            transArray = (object[])transparentArray;

            status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, true, 1, null, 0);
            status = swModelDocExt.SelectByID2("Plane2", "PLANE", 0, 0, 0, true, 2, null, 0);
            status = swModelDocExt.SelectByID2("Plane3", "PLANE", 0, 0, 0, true, 4, null, 0);

            swModelViewManager = (ModelViewManager)swModel.ModelViewManager;
            swSectionViewData = swModelViewManager.CreateSectionViewData();

            swSectionViewData.FirstColor = 16711680;
            swSectionViewData.SecondColor = 65280;
            swSectionViewData.ThirdColor = 255;
            swSectionViewData.ShowSectionCap = true;
            swSectionViewData.KeepCapColor = true;
            swSectionViewData.GraphicsOnlySection = true;

            swSectionViewData.ZonalSection = true;

            swSectionViewData.SelectiveSection = true;
            swSectionViewData.SelectiveSectioningList = selArray;
        }
    }
}
```

```

swSectionViewData.SectionedZones = 16; //swZonalSectionViewZones_e.swZonalSectionViewZones_swZonalSectionViewZone_5
swSectionViewData.ExcludeSelectedItems = true;

swSectionViewData.TransparentSection = true;
swSectionViewData.TransparencyList = transArray;
swSectionViewData.SectionTransparentItemsTransparent = true;
swSectionViewData.TransparencyValue = 0.9;

status = swModelViewManager.CreateSectionView(swSectionViewData);

swModel.ClearSelection2(true);
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}

```

Selectively and Transparently Section a Section View Example (VBA)

This example shows how to selectively and transparently section a section view.

```

'-----
' Preconditions: Verify that the assembly to open exists.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects the component for selective sectioning.
' 3. Selects the components for transparent sectioning.
' 4. Selects the planes for sectioning.
' 5. Creates the SectionViewData object.
'   a. Sets the section method to zonal.
'   b. Sets to cap the sections.
'   c. Sets to generate a graphics-only section view.
'   d. Enables selective sectioning.
'       1. Sets the component selected in step 2 for selective
'          sectioning.
'       2. Sets the intersection zone.
'       3. Sets to exclude the component set in step 5.d.1 from
'          selective sectioning.
'   e. Enables transparent sectioning.
'       1. Sets the components selected in step 3
'          for transparent sectioning.
'       2. Sets to include the sectioned components set in step 5.e.1
'          in transparent sectioning.
'       3. Sets the level of transparency.
' 6. Creates the section view.
' 7. Examine the graphics area.
' 8. Click Section View twice in the Heads-Up View toolbar and
'    examine the Section View PropertyManager page.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim selectiveArray(0) As SldWorks.Component2
Dim selComponent As SldWorks.Component2
Dim swSelMgr As SldWorks.SelectionMgr
Dim transparentArray(1) As SldWorks.Component2
Dim transComponent As SldWorks.Component2
Dim transComponent2 As SldWorks.Component2
Dim swSectionViewData As SldWorks.SectionViewData
Dim swModelViewManager As SldWorks.ModelViewManager
Dim transArray As Variant
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim selArray As Variant

Sub main()

    Set swApp = Application.SldWorks
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\floxpress\ball valve\ball_valve.sldasm"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension

    status = swModelDocExt.SelectByID2("Ball-1@ball_valve", "COMPONENT", 0, 0, 0, True, 8, Nothing, 0)
    Set swSelMgr = swModel.SelectionManager
    Set selComponent = swSelMgr.GetSelectedObject6(1, -1)
    Set selectiveArray(0) = selComponent
    selArray = selectiveArray

    status = swModelDocExt.SelectByID2("Side-1@ball_valve", "COMPONENT", 0, 0, 0, True, 32, Nothing, 0)
    status = swModelDocExt.SelectByID2("Side-2@ball_valve", "COMPONENT", 0, 0, 0, True, 32, Nothing, 0)
    Set transComponent = swSelMgr.GetSelectedObject6(1, -1)
    Set transComponent2 = swSelMgr.GetSelectedObject6(2, -1)
    Set transparentArray(0) = transComponent
    Set transparentArray(1) = transComponent2
    transArray = transparentArray

    status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("Plane2", "PLANE", 0, 0, 0, True, 2, Nothing, 0)
    status = swModelDocExt.SelectByID2("Plane3", "PLANE", 0, 0, 0, True, 4, Nothing, 0)

    Set swModelViewManager = swModel.ModelViewManager
    Set swSectionViewData = swModelViewManager.CreateSectionViewData()

    swSectionViewData.FirstColor = 16711680
    swSectionViewData.SecondColor = 65280
    swSectionViewData.ThirdColor = 255
    swSectionViewData.ShowSectionCap = True
    swSectionViewData.KeepCapColor = True
    swSectionViewData.GraphicsOnlySection = True

    swSectionViewData.ZonalSection = True

    swSectionViewData.SelectiveSection = True
    swSectionViewData.SelectiveSectioningList = selArray
    swSectionViewData.SectionedZones = 16 'swZonalSectionViewZones_e.swZonalSectionViewZones_swZonalSectionViewZone_5

```

```
swSectionViewData.ExcludeSelectedItems = True

swSectionViewData.TransparentSection = True
swSectionViewData.TransparencyList = transArray
swSectionViewData.SectionTransparentItemsTransparent = True
swSectionViewData.TransparencyValue = 0.9

status = swModelViewManager.CreateSectionView(swSectionViewData)

swModel.ClearSelection2 True

End Sub
```

Selectively and Transparently Section a Section View Example (VB.NET)

This example shows how to selectively and transparently section a section view.

```

'-----
' Preconditions: Verify that the assembly to open exists.
'
' Postconditions:
' 1. Opens the assembly.
' 2. Selects the component for selective sectioning.
' 3. Selects the components for transparent sectioning.
' 4. Selects the planes for sectioning.
' 5. Creates the SectionViewData object.
'   a. Sets the section method to zonal.
'   b. Sets to cap the sections.
'   c. Sets to generate a graphics-only section view.
'   d. Enables selective sectioning.
'       1. Sets the component selected in step 2 for selective
'          sectioning.
'       2. Sets the intersection zone.
'       3. Sets to exclude the component set in step 5.d.1 from
'          selective sectioning.
'   e. Enables transparent sectioning.
'       1. Sets the components selected in step 3
'          for transparent sectioning.
'       2. Sets to include the sectioned components set in step 5.e.1
'          in transparent sectioning.
'       3. Sets the level of transparency.
' 6. Creates the section view.
' 7. Examine the graphics area.
' 8. Click Section View twice in the Heads-Up View toolbar and
'    examine the Section View PropertyManager page.
'
' NOTE: Because the assembly is used elsewhere, do not save changes.
'-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System

```

```
Partial Class SolidWorksMacro
```

```
    Public Sub main()
```

```

        Dim swModel As ModelDoc2
        Dim swModelDocExt As ModelDocExtension
        Dim selectiveArray(0) As Component2
        Dim selComponent As Component2
        Dim swSelMgr As SelectionMgr
        Dim transparentArray(1) As Component2
        Dim transComponent As Component2
        Dim transComponent2 As Component2
        Dim swSectionViewData As SectionViewData
        Dim swModelViewManager As ModelViewManager
        Dim transArray As Object
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim selArray As Object

```

```

        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\floxpress\ball_valve\ball_valve.sldasm"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocASSEMBLY, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension

```

```

        status = swModelDocExt.SelectByID2("Ball-1@ball_valve", "COMPONENT", 0, 0, 0, True, 8, Nothing, 0)
        swSelMgr = swModel.SelectionManager
        selComponent = swSelMgr.GetSelectedObject6(1, -1)
        selectiveArray(0) = selComponent
        selArray = selectiveArray

```

```

        status = swModelDocExt.SelectByID2("Side-1@ball_valve", "COMPONENT", 0, 0, 0, True, 32, Nothing, 0)
        status = swModelDocExt.SelectByID2("Side-2@ball_valve", "COMPONENT", 0, 0, 0, True, 32, Nothing, 0)
        transComponent = swSelMgr.GetSelectedObject6(1, -1)
        transComponent2 = swSelMgr.GetSelectedObject6(2, -1)
        transparentArray(0) = transComponent
        transparentArray(1) = transComponent2
        transArray = transparentArray

```

```

        status = swModelDocExt.SelectByID2("Plane1", "PLANE", 0, 0, 0, True, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("Plane2", "PLANE", 0, 0, 0, True, 2, Nothing, 0)
        status = swModelDocExt.SelectByID2("Plane3", "PLANE", 0, 0, 0, True, 4, Nothing, 0)

```

```

        swModelViewManager = swModel.ModelViewManager
        swSectionViewData = swModelViewManager.CreateSectionViewData()

```

```

        swSectionViewData.FirstColor = 16711680
        swSectionViewData.SecondColor = 65280
        swSectionViewData.ThirdColor = 255
        swSectionViewData.ShowSectionCap = True
        swSectionViewData.KeepCapColor = True
        swSectionViewData.GraphicsOnlySection = True

```

```
        swSectionViewData.ZonalSection = True
```

```

        swSectionViewData.SelectiveSection = True
        swSectionViewData.SelectiveSectioningList = selArray
        swSectionViewData.SectionedZones = 16 'swZonalSectionViewZones_e.swZonalSectionViewZones_swZonalSectionViewZone_5
        swSectionViewData.ExcludeSelectedItems = True

```

```

        swSectionViewData.TransparentSection = True
        swSectionViewData.TransparencyList = transArray

```

```
swSectionViewData.SectionTransparentItemsTransparent = True
swSectionViewData.TransparencyValue = 0.9
status = swModelViewManager.CreateSectionView(swSectionViewData)
```

```
swModel.ClearSelection2(True)
```

```
End Sub
```

```
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
```

```
End Class
```

Set Active Display State of Referenced Component Example (C#)

This example shows how to set the active display state of a referenced component.

```
//-----
// Preconditions:
// 1. Open public_documents\samples\tutorial\driveworksexpress\mobile gantry.sldasm.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Selects leg<1>.
// 2. Gets the selected referenced component's active display state.
// 3. Sets the selected referenced component's active display state.
// 4. Examine the Immediate window, FeatureManager design tree,
//    and graphics area.
//
// NOTE: Because the assembly is used elsewhere, do not
// save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;

namespace Macro1.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            ModelDocExtension swModelDocExtension = default(ModelDocExtension);
            Component2 swComponent = default(Component2);
            SelectionMgr swSelectionManager = default(SelectionMgr);
            bool status = false;

            swModel = (ModelDoc2)swApp.ActiveDoc;
            swModelDocExtension = (ModelDocExtension)swModel.Extension;
            status = swModelDocExtension.SelectByID("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, false, 0, null, 0);
            swSelectionManager = (SelectionMgr)swModel.SelectionManager;
            swComponent = (Component2)swSelectionManager.GetSelectedObjectsComponent4(1, -1);
            Debug.Print("Get active display state: " + swComponent.ReferencedDisplayState2);
            swComponent.ReferencedDisplayState2 = "<Default<As Machined>> Display State 1";
            Debug.Print("Set active display state: " + swComponent.ReferencedDisplayState2);

        }

        /// <summary>
        /// The SldWorks swApp variable is pre-assigned for you.
        /// </summary>
        public SldWorks swApp;
    }
}
```

Set Active Display State of Referenced Component Example (VBA)

This example shows how to set the active display state of a referenced component.

```
'-----  
' Preconditions:  
' 1. Open public_documents\samples\tutorial\driveworksxpess\mobile gantry.sldasm.  
' 2. Open the Immediate window.  
'  
' Postconditions:  
' 1. Selects leg<1>.  
' 2. Gets the selected referenced component's active display state.  
' 3. Sets the selected referenced component's active display state.  
' 4. Examine the Immediate window, FeatureManager design tree,  
'    and graphics area.  
'  
' NOTE: Because the assembly is used elsewhere, do not  
'       save changes.  
'-----  
Option Explicit  
  
Dim swApp As SldWorks.SldWorks  
Dim swModel As SldWorks.ModelDoc2  
Dim swModelDocExtension As SldWorks.ModelDocExtension  
Dim swComponent As SldWorks.Component2  
Dim swSelectionManager As SldWorks.SelectionMgr  
Dim status As Boolean  
  
Sub main()  
    Set swApp = Application.SldWorks  
    Set swModel = swApp.ActiveDoc  
    Set swModelDocExtension = swModel.Extension  
    status = swModelDocExtension.SelectByID("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)  
    Set swSelectionManager = swModel.SelectionManager  
    Set swComponent = swSelectionManager.GetSelectedObjectsComponent4(1, -1)  
    Debug.Print "Get active display state: " & swComponent.ReferencedDisplayState2  
    swComponent.ReferencedDisplayState2 = "<Default<As Machined>>_Display State 1"  
    Debug.Print "Set active display state: " & swComponent.ReferencedDisplayState2  
  
End Sub
```

Set Active Display State of Referenced Component Example (VB.NET)

This example shows how to set the active display state of a referenced component.

```

-----
Preconditions:
1. Open public_documents\samples\tutorial\driveworksexpress\mobile gantry.sldasm.
2. Open the Immediate window.

Postconditions:
1. Selects leg<1>.
2. Gets the selected referenced component's active display state.
3. Sets the selected referenced component's active display state.
4. Examine the Immediate window, FeatureManager design tree,
   and graphics area.

NOTE: Because the assembly is used elsewhere, do not
save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swModelDocExtension As ModelDocExtension
        Dim swComponent As Component2
        Dim swSelectionManager As SelectionMgr
        Dim status As Boolean

        swModel = swApp.ActiveDoc
        swModelDocExtension = swModel.Extension
        status = swModelDocExtension.SelectByID2("leg-1@mobile gantry", "COMPONENT", 0, 0, 0, False, 0, Nothing, 0)
        swSelectionManager = swModel.SelectionManager
        swComponent = swSelectionManager.GetSelectedObjectsComponent4(1, -1)
        Debug.Print("Get active display state: " & swComponent.ReferencedDisplayState2)
        swComponent.ReferencedDisplayState2 = "<Default<As Machined>>_Display State 1"
        Debug.Print("Set active display state: " & swComponent.ReferencedDisplayState2)

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```

Set All Assembly Components Lightweight or Resolved Example (VBA)

This example shows how to set all assembly components to either lightweight or resolved.

```
'-----
' Preconditions:
' 1. Open an assembly.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Sets all assembly components to either
'    lightweight or fully resolved.
' 2. Examine the FeatureManager design tree
'    and Immediate window.
'-----
```

Option Explicit

```
Sub ProcessComponent(swApp As SldWorks.SldWorks, swModel As SldWorks.ModelDoc2, swComp As SldWorks.Component2, swComponentSuppressionState As
swComponentSuppressionState_e, sPadStr As String)
```

```
    Dim vChildCompArr As Variant
    Dim vChildComp As Variant
    Dim swChildComp As SldWorks.Component2
    Dim nRetVal As Long

    vChildCompArr = swComp.Children
    For Each vChildComp In vChildCompArr
        Set swChildComp = vChildComp
        nRetVal = swChildComp.SetSuppression2(swComponentSuppressionState)
        Debug.Print sPadStr & swChildComp.Name2 & " <" & swChildComp.ReferencedConfiguration & "> --> " & swChildComp.GetPathName
        ProcessComponent swApp, swModel, swChildComp, swComponentSuppressionState, sPadStr + " "
    Next vChildComp
```

End Sub

```
Sub main()
```

```
    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim swFeatMgr As SldWorks.FeatureManager
    Dim swConfigMgr As SldWorks.ConfigurationManager
    Dim swConfig As SldWorks.Configuration
    Dim swRootComp As SldWorks.Component2
    Dim bRet As Boolean
    Dim nSuppressState As Long
    Dim nResponse As Long

    nResponse = MsgBox("Set all components to lightweight or resolved (Yes = Lightweight; No = Resolved)?", vbYesNo)
    If nResponse = vbYes Then
        nSuppressState = swComponentLightweight
    Else
        nSuppressState = swComponentFullyResolved
    End If

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc
    Set swFeatMgr = swModel.FeatureManager
    Set swConfigMgr = swModel.ConfigurationManager
    Set swConfig = swConfigMgr.ActiveConfiguration
    Set swRootComp = swConfig.GetRootComponent3(True)
    swFeatMgr.EnableFeatureTree = False

    Debug.Print "File = " & swModel.GetPathName

    ProcessComponent swApp, swModel, swRootComp, nSuppressState, " "

    swFeatMgr.EnableFeatureTree = True
```

End Sub

Set and Get Maximum Extension Line Length Example (VBA)

This example shows how to set and get the maximum extension line length for a display dimension.

```
'-----
' Preconditions:
' 1. Open public_documents\samples\tutorial\lesson2\tutor1.sldprt.
' 2. Right-click Annotations and click Show Feature Dimensions.
' 3. Open the Immediate window.
'
' Postconditions:
' 1. Selects a dimension.
' 2. Sets the maximum length for the selected dimension's
'    witness extension lines to the specified value.
' 3. Examine the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModelDoc As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelMgr As SldWorks.SelectionMgr
Dim swDisplayDim As SldWorks.DisplayDimension
Dim swModelView As SldWorks.ModelView
Dim selType As Long
Dim maxLineLength As Double
Dim status As Boolean
Dim rect As Variant

Sub main()

    Set swApp = Application.SldWorks
    Set swModelDoc = swApp.ActiveDoc
    Set swModelDocExt = swModelDoc.Extension
    Set swSelMgr = swModelDoc.SelectionManager
    status = swModelDocExt.SelectByID("D2@Sketch2@tutor1.SLDPRT", "DIMENSION", 2.29248368227093E-03, 6.41116623573457E-02, 5.51580671328052E-02, False,
0, Nothing, 0)
    selType = swSelMgr.GetSelectedObjectType3(1, -1)
    If selType = swSelDIMENSIONS Then
        Set swDisplayDim = swSelMgr.GetSelectedObject6(1, -1)
        swDisplayDim.MaxWitnessLineLength = 130
    End If
    maxLineLength = swDisplayDim.MaxWitnessLineLength
    Debug.Print "New maximum witness extension line length: " & maxLineLength

    Set swModelView = swModelDoc.ActiveView
    Set rect = Nothing
    swModelView.GraphicsRedraw (rect)

End Sub
```

Set and Get Sheet Metal Part's Persistent Reference IDs Example (C#)

This example shows how to set and get persistent reference IDs (PIDs) on a sheet metal part.

The entities in flattened and unflattened (folded) states of sheet metal in SOLIDWORKS Design do not have the same properties, making it difficult to track entities across states of sheet metal.

- [IModelDocExtension::GetFlattenSheetMetalPersistReference](#) returns a PID array for an entity in the flattened state of the sheet metal part.
- [IModelDocExtension::GetSheetMetalObjectsByPersistReference](#) uses the PID byte array to retrieve an array of objects that comprise the previously selected entity in the folded state of the part.

Together these methods provide a way to track entities across sheet metal states.

```
//-----
// Preconditions:
// 1. Verify that the specified part exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens specified part document.
// 2. Unsuppresses the Flat-Pattern1 feature.
// 3. Gets the PIDs of the selected face on the
// Flat-Pattern1 feature.
// 4. Suppresses the Flat-Pattern1 feature.
// 5. Uses the PIDs to retrieve and highlight the
// array of objects that comprise the previously
// selected face in the folded state of the part.
// 6. Examine the Immediate window, graphics area, and
// FeatureManager design tree.
//
// NOTE: Because the part document is used elsewhere,
// do not save changes.
//-----

using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System;
using System.Diagnostics;

namespace EditSuppress2.csproj
{
    partial class SolidWorksMacro
    {
        ModelDoc2 swModel;
        ModelDocExtension swModelDocExt;
        SelectionMgr swSelectionMgr;
        Face2 swFace;
        SelectData swSelectData;
        byte[] pid;
        object selObj;
        object[] swObjList;
        string fileName;
        int errors;
        int warnings;
        bool boolstatus;
        int i;
        int j;

        public void Main()
        {
            // Open the specified sheet metal part
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\api\\2012-sm.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);

            // Get flat-pattern feature
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            boolstatus = swModelDocExt.SelectByID2("Flat-
Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);

            // Unsuppress (unfold) flat-pattern feature
            boolstatus = swModel.EditUnsuppress2();

            // Select the top face on the flattened sheet metal part
            boolstatus = swModelDocExt.SelectByID2("", "FACE", -3.46526388559596E-02, 0, 0.011695220708134, false, 0, null, 0)
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swFace = (Face2)swSelectionMgr.GetSelectedObject6(1, -1);

            // Get the persistent reference IDs for the
            // the selected face on the flattened sheet
            // metal part
            pid = (byte[])swModelDocExt.GetFlattenSheetMetalPersistReference(swFace);

            // Print the PID values to the Immediate window
            for (j = pid.GetLowerBound(0); j <= pid.GetUpperBound(0); j++)
            {
                Debug.Print("PID = " + pid[j]);
            }

            // Suppress (fold) flat-pattern feature
            boolstatus = swModelDocExt.SelectByID2("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            swModel.EditSuppress2();

            // Use the persistent reference IDs to
            // retrieve an array of objects that comprise
            // the entity in the folded state of the part
            swObjList = (object[])swModelDocExt.GetSheetMetalObjectsByPersistReference((pid), out errors);

            if ((swObjList != null))
        }
    }
}
```

```

    {
        swModel.ClearSelection2(true);

        for (i = swObjList.GetLowerBound(0); i <= swObjList.GetUpperBound(0); i++)
        {
            selObj = swObjList[i];

            if (selObj == null)
            {
                Debug.Print("Persistent reference ID conversion error.");
                return;
            }
            else
            {
                SelectObject(selObj, true);
            }
        }

        Debug.Print("The entities that comprise the previously selected entity in the folded state are selected.");
    }

private void SelectObject(object selObj, bool append)
{
    if (selObj is Entity)
    {
        Entity selObj1;
        selObj1 = (Entity)selObj;
        swSelectData = (SelectData)swSelectionMgr.CreateSelectData();
        selObj1.Select4(append, swSelectData);
    }
    else
    {
        Debug.Print("Need selection handle.");
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
}
}

```

Set and Get Sheet Metal Part's Persistent Reference IDs Example (VBA)

This example shows how to set and get persistent reference IDs (PIDs) on a sheet metal part.

The entities in flattened and unflattened (folded) states of sheet metal in SOLIDWORKS Design do not have the same properties, making it difficult to track entities across states of sheet metal.

- [IModelDocExtension::GetFlattenSheetMetalPersistReference](#) returns a PID array for an entity in the flattened state of the sheet metal part.
- [IModelDocExtension::GetSheetMetalObjectsByPersistReference](#) uses the PID byte array to retrieve an array of objects that comprise the previously selected entity in the folded state of the part.

Together these methods provide a way to track entities across sheet metal states.

```
'-----
' Preconditions:
' 1. Verify that specified part exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens specified part document.
' 2. Unsuppresses the Flat-Pattern1 feature.
' 3. Gets the PIDs of the selected face on the
'    Flat-Pattern1 feature.
' 4. Suppresses the Flat-Pattern1 feature.
' 5. Uses the PIDs to retrieve and highlight the
'    array of objects that comprise the previously
'    selected face in the folded state of the part.
' 6. Examine the Immediate window, graphics area, and
'    FeatureManager design tree.
'
' NOTE: Because the part is used elsewhere,
' do not save changes.
'-----
```

Option Explicit

```
Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim swFace As SldWorks.Face2
Dim swSelectData As SldWorks.SelectData
Dim pid As Variant
Dim selObj As Object
Dim swObjList As Variant
Dim errors As Long
Dim warnings As Long
Dim boolstatus As Boolean
Dim i As Long
Dim j As Long

Sub main()

    Set swApp = Application.SldWorks

    ' Open part document
    Set swModel = swApp.OpenDoc6("C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt", swDocPART, swOpenDocOptions_Silent, "", errors, warnings)

    ' Get flat-pattern feature
    Set swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)

    ' Unsuppress (unfold) flat-pattern feature
    boolstatus = swModel.EditUnsuppress2

    ' Select the top face on the flattened sheet metal part
    boolstatus = swModelDocExt.SelectByID("", "FACE", -3.46526388559596E-02, 0, 0.011695220708134, False, 0, Nothing, 0)
    Set swSelectionMgr = swModel.SelectionManager
    Set swFace = swSelectionMgr.GetSelectedObject6(1, -1)

    ' Get the persistent reference IDs for the
    ' the selected face on the flattened sheet
    ' metal part
    pid = swModelDocExt.GetFlattenSheetMetalPersistReference(swFace)

    ' Print the PID values to the Immediate window
    j = 0
    For j = LBound(pid) To UBound(pid)
        Debug.Print ("PID = " & pid(j))
    Next j

    ' Suppress (fold) flat-pattern feature
    boolstatus = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    swModel.EditSuppress2

    ' Use the persistent reference IDs to
    ' retrieve and highlight an array of objects
    ' that comprise the previously selected entity in
    ' the folded state of the part
    swObjList = swModelDocExt.GetSheetMetalObjectsByPersistReference((pid), errors)

    If Not (IsEmpty(swObjList)) Then
        swModel.ClearSelection2 (True)
    End If
End Sub
```

```

    For i = LBound(swObjList) To UBound(swObjList)
        Set selObj = swObjList(i)
        If selObj Is Nothing Then
            Debug.Print "PID conversion error."
            Exit Sub
        Else
            SelectObject selObj, True
        End If
    Next i
End If

Debug.Print "The entities that comprise the previously selected entity in the folded state are selected."

End Sub

Private Sub SelectObject(selObj As Object, append As Boolean)
    If TypeOf selObj Is SldWorks.Entity Then
        Set swSelectData = swSelectionMgr.CreateSelectData
        selObj.Select4 append, swSelectData
    Else
        Debug.Print ("Need selection handle.")
    End If
End Sub

```

Set and Get Sheet Metal Part's Persistent Reference IDs Example (VB.NET)

This example shows how to set and get persistent reference IDs (PIDs) on a sheet metal part.

The entities in flattened and unflattened (folded) states of sheet metal in SOLIDWORKS Design do not have the same properties, making it difficult to track entities across states of sheet metal.

- [IModelDocExtension.GetFlattenSheetMetalPersistReference](#) returns a PID array for an entity in the flattened state of the sheet metal part.
- [IModelDocExtension.GetSheetmetalObjectsByPersistReference](#) uses the PID byte array to retrieve an array of objects that comprise the previously selected entity in the folded state of the part.

Together these methods provide a way to track entities across sheet metal states.

```

-----
Preconditions:
1. Verify that the specified part exists.
2. Open the Immediate window.

Postconditions:
1. Opens specified part document.
2. Unsuppresses the Flat-Pattern1 feature.
3. Gets the PIDs of the selected face on the
   Flat-Pattern1 feature.
4. Suppresses the Flat-Pattern1 feature.
5. Uses the PIDs to retrieve and highlight the
   array of objects that comprise the previously
   selected face in the folded state of the part.
6. Examine the Immediate window, graphics area, and
   FeatureManager design tree.

NOTE: Because the part is used elsewhere,
do not save any changes when closing it.
-----

```

```

Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System
Imports System.Diagnostics

```

```
Partial Class SolidWorksMacro
```

```

    Dim swModel As ModelDoc2
    Dim swModelDocExt As ModelDocExtension
    Dim swSelectionMgr As SelectionMgr
    Dim swFace As Face2
    Dim swSelectData As SelectData
    Dim pid As Object
    Dim FileName As String
    Dim selObj As Object
    Dim swObjList As Object
    Dim errors As Integer
    Dim warnings As Integer
    Dim boolstatus As Boolean
    Dim i As Integer
    Dim j As Integer

```

```
Sub main()
```

```

    ' Open the specified sheet metal part
    FileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\api\2012-sm.sldprt"
    swModel = swApp.OpenDoc6(FileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)

    ' Get flat-pattern feature
    swModelDocExt = swModel.Extension
    boolstatus = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)

    ' Unsuppress (unfold) flat-pattern feature
    boolstatus = swModel.EditUnsuppress2

    ' Select the top face on the flattened sheet metal part
    boolstatus = swModelDocExt.SelectByID("", "FACE", -3.46526388559596E-02, 0, 0.011695220708134, False, 0, Nothing, 0)
    swSelectionMgr = swModel.SelectionManager
    swFace = swSelectionMgr.GetSelectedObject6(1, -1)

    ' Get the persistent reference IDs for the
    ' the selected face on the flattened sheet
    ' metal part
    pid = swModelDocExt.GetFlattenSheetMetalPersistReference(swFace)

    ' Print the PID values to the Immediate window
    j = 0
    For j = LBound(pid) To UBound(pid)
        Debug.Print("PID = " & pid(j))
    Next j

    ' Suppress (fold) flat-pattern feature
    boolstatus = swModelDocExt.SelectByID("Flat-Pattern1", "BODYFEATURE", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
    swModel.EditSuppress2()

    ' Use the persistent reference IDs to
    ' retrieve an array of objects that comprise
    ' the previously select entity in the folded state of the part
    swObjList = swModelDocExt.GetSheetmetalObjectsByPersistReference((pid), errors)
    If Not swObjList Is Nothing Then
        swModel.ClearSelection2(True)
        For i = LBound(swObjList) To UBound(swObjList)
            selObj = swObjList(i)
            If selObj Is Nothing Then
                Debug.Print("PID conversion error.")
                Exit Sub
            Else
                SelectObject(selObj, True)
            End If
        Next i
    End If

    Debug.Print("The entities that comprise the previously selected entity in the folded state are selected.")

```

```
End Sub
```

```

Private Sub SelectObject(ByVal selObj As Object, ByVal append As Boolean)
    If TypeOf selObj Is Entity Then

```

```
        swSelectData = swSelectionMgr.CreateSelectData
        selObj.Select4(append, swSelectData)
    Else
        Debug.Print("Need selection handle.")
    End If
End Sub
''' <summary>
''' The SldWorks swApp variable is pre-assigned for you.
''' </summary>
Public swApp As SldWorks
End Class
```

Set Blocking State Example (VBA)

This example shows how to set and reset the blocking state for a model document.

```
'-----
' Preconditions:
' 1. Open a model document
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Displays a message box asking whether
'    to reset the blocking state to the previous
'    blocking state.
' 2. Click Yes to reset the blocking state to allow changes
'    to the document.
' 3. Examine the Immediate window and click the Sketch tab to verify
'    that you can make changes to the document.
' 4. Run the macro again and click No to set the blocking state
'    to disallow changes to document.
' 5. Examine the Immediate window and the Sketch toolbar to verify
'    that you cannot make changes to the document.
' 6. Run the macro again and click Yes.
' 7. Examine the Immediate window and the Sketch toolbar to verify
'    that you can make changes to the document.
'-----
Option Explicit

Sub main()

    Dim swApp As SldWorks.SldWorks
    Dim swModel As SldWorks.ModelDoc2
    Dim nResponse As Integer
    Dim bRet As Boolean

    Set swApp = Application.SldWorks
    Set swModel = swApp.ActiveDoc

    ' Get current blocking state
    Debug.Print "Starting blocking state = " & swModel.GetBlockingState

    ' Ask user if blocking state should be reset to previous state
    nResponse = MsgBox("Reset blocking state?", vbYesNo)

    ' If user responds yes, then reset blocking state
    If nResponse = vbYes Then
        swModel.ResetBlockingState
    Else
        ' If use responds no, then set blocking state to
        ' disallow changes to document
        swModel.SetBlockingState swPartialModifyBlock
    End If

    Debug.Print "Ending blocking state = " & swModel.GetBlockingState
    Debug.Print ""

End Sub
```

Set Bodies for Move/Copy Example (C#)

This example shows how to set a body for a move/copy.

```
//-----
// Preconditions:
// 1. Verify that the specified part document exists.
// 2. Open the Immediate window.
//
// Postconditions:
// 1. Opens the specified part document.
// 2. Selects a solid body and two vertices.
// 3. Inserts a move/copy body feature.
// 4. Sets the body to move, copy, or rotate.
// 5. Examine the FeatureManager design tree, the graphics area, and
//    the Immediate window.
//
// NOTE: Because the part is used elsewhere, do not save changes.
//-----
using SolidWorks.Interop.sldworks;
using SolidWorks.Interop.swconst;
using System.Runtime.InteropServices;
using System;
using System.Diagnostics;
using System.Collections;

namespace Macro1CSharp.csproj
{
    public partial class SolidWorksMacro
    {
        public void Main()
        {
            ModelDoc2 swModel = default(ModelDoc2);
            PartDoc swPart = default(PartDoc);
            ModelDocExtension swModelDocExt = default(ModelDocExtension);
            FeatureManager swFeatureManager = default(FeatureManager);
            Feature swFeature = default(Feature);
            MoveCopyBodyFeatureData swMoveCopyBodyFeatureData = default(MoveCopyBodyFeatureData);
            SelectionMgr swSelectionMgr = default(SelectionMgr);
            bool status = false;
            int errors = 0;
            int warnings = 0;
            string fileName = null;
            object[] bodyArr = new object[1];
            Body2 aBody = default(Body2);

            //Open part document
            fileName = "C:\\Users\\Public\\Documents\\SOLIDWORKS\\SOLIDWORKS 2018\\samples\\tutorial\\multibody\\multi_inter.sldprt";
            swModel = (ModelDoc2)swApp.OpenDoc6(fileName, (int)swDocumentTypes_e.swDocPART, (int)swOpenDocOptions_e.swOpenDocOptions_Silent, "", ref error
s, ref warnings);
            swModelDocExt = (ModelDocExtension)swModel.Extension;
            swFeatureManager = (FeatureManager)swModel.FeatureManager;
            swSelectionMgr = (SelectionMgr)swModel.SelectionManager;
            swPart = (PartDoc)swModel;

            //Select solid body and vertices for move/copy body feature
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, true, 0, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, true, 0, null, 0);
            swModel.ClearSelection2(true);
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, false, 1, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, true, 4, null, 0);
            status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, true, 8, null, 0);

            //Insert move/copy body feature
            swFeature = (Feature)swFeatureManager.InsertMoveCopyBody2(0, 0, 0, 0, -0.085, 0, 0.065, 0, 0, 0,
            true, 1);
            swFeature = (Feature)swPart.FeatureByName("Body-Move/Copy1");

            //Roll forward the feature and get and set move/copy body feature data
            swMoveCopyBodyFeatureData = (MoveCopyBodyFeatureData)swFeature.GetDefinition();
            status = swMoveCopyBodyFeatureData.AccessSelections(swModel, null);

            //Get the body to set
            status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, false, 0, null, (int)swSelectOption_e.swSelectOptionDefault);
            bodyArr[0] = (object)swSelectionMgr.GetSelectedObject6(1, -1);
            swModel.ClearSelection2(true);
            swMoveCopyBodyFeatureData.Bodies = bodyArr[0];
            if ((bodyArr == null))
            {
                Debug.Print("Body not set.");
            }
            else
            {
                Debug.Print("Body set.");
                aBody = (Body2)bodyArr[0];
                Debug.Print("Name of body set: " + aBody.Name);
            }

            //Roll back the feature
            swMoveCopyBodyFeatureData.ReleaseSelectionAccess();

            swModel.ViewZoomtofit2();
        }
    }
}

/// <summary>
/// The SldWorks swApp variable is pre-assigned for you.
/// </summary>
public SldWorks swApp;
```

Set Bodies for Move/Copy Example (VBA)

This example shows how to set a body for a move/copy.

```
'-----
' Preconditions:
' 1. Verify that the specified part document exists.
' 2. Open the Immediate window.
'
' Postconditions:
' 1. Opens the specified part document.
' 2. Selects a solid body and two vertices.
' 3. Inserts a move/copy body feature.
' 4. Sets the body to move, copy, or rotate.
' 5. Examine the FeatureManager design tree, the graphics area, and
'    the Immediate window.
'
' NOTE: Because the part is used elsewhere, do not save changes.
'-----
Option Explicit

Dim swApp As SldWorks.SldWorks
Dim swModel As SldWorks.ModelDoc2
Dim swPart As SldWorks.PartDoc
Dim swModelDocExt As SldWorks.ModelDocExtension
Dim swFeatureManager As SldWorks.FeatureManager
Dim swFeature As SldWorks.Feature
Dim swMoveCopyBodyFeatureData As SldWorks.MoveCopyBodyFeatureData
Dim swSelectionMgr As SldWorks.SelectionMgr
Dim status As Boolean
Dim errors As Long
Dim warnings As Long
Dim fileName As String
Dim bodyArr(0) As Object
Dim aBody As SldWorks.Body2

Sub main()

    Set swApp = Application.SldWorks

    'Open part document
    fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS 2018\samples\tutorial\multibody\multi_inter.sldprt"
    Set swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
    Set swModelDocExt = swModel.Extension
    Set swFeatureManager = swModel.FeatureManager
    Set swSelectionMgr = swModel.SelectionManager
    Set swPart = swModel

    'Select solid body and vertices for move/copy body feature
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)
    swModel.ClearSelection2 True
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)
    status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)

    'Insert move/copy body feature
    Set swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)
    Set swFeature = swPart.FeatureByName("Body-Move/Copy1")

    'Roll forward the feature and get and set move/copy body feature data
    Set swMoveCopyBodyFeatureData = swFeature.GetDefinition
    status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)

    'Get the body to set
    status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, swSelectOptionDefault)
    Set bodyArr(0) = swSelectionMgr.GetSelectedObject6(1, -1)
    swModel.ClearSelection2 True
    swMoveCopyBodyFeatureData.bodies = bodyArr(0)
    If IsEmpty(bodyArr) Then
        Debug.Print "Body not set."
    Else
        Debug.Print "Body set."
        aBody = bodyArr(0)
        Debug.Print "Name of body set: " & aBody.Name
    End If

    'Roll back the feature
    swMoveCopyBodyFeatureData.ReleaseSelectionAccess

    swModel.ViewZoomtofit2

End Sub
```

Set Bodies for Move/Copy Example (VB.NET)

This example shows how to a body for a move/copy.

```

-----
Preconditions:
1. Verify that the specified part document exists.
2. Open the Immediate window.
Postconditions:
1. Opens the specified part document.
2. Selects a solid body and two vertices.
3. Inserts a move/copy body feature.
4. Sets the body to move, copy, or rotate.
5. Examine the FeatureManager design tree, the graphics area, and
   the Immediate window.
-----
NOTE: Because the part is used elsewhere, do not save changes.
-----
Imports SolidWorks.Interop.sldworks
Imports SolidWorks.Interop.swconst
Imports System.Runtime.InteropServices
Imports System
Imports System.Diagnostics

Partial Class SolidWorksMacro

    Public Sub main()

        Dim swModel As ModelDoc2
        Dim swPart As PartDoc
        Dim swModelDocExt As ModelDocExtension
        Dim swFeatureManager As FeatureManager
        Dim swFeature As Feature
        Dim swMoveCopyBodyFeatureData As MoveCopyBodyFeatureData
        Dim swSelectionMgr As SelectionMgr
        Dim status As Boolean
        Dim errors As Integer
        Dim warnings As Integer
        Dim fileName As String
        Dim bodyArr(0) As Object
        Dim aBody As Body2

        'Open part document
        fileName = "C:\Users\Public\Documents\SOLIDWORKS\SOLIDWORKS_2018\samples\tutorial\multibody\multi_inter.sldprt"
        swModel = swApp.OpenDoc6(fileName, swDocumentTypes_e.swDocPART, swOpenDocOptions_e.swOpenDocOptions_Silent, "", errors, warnings)
        swModelDocExt = swModel.Extension
        swFeatureManager = swModel.FeatureManager
        swSelectionMgr = swModel.SelectionManager
        swPart = swModel

        'Select solid body and vertices for move/copy body feature
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 0, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 0, Nothing, 0)
        swModel.ClearSelection2(True)
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 1, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, 0, 0.065, True, 4, Nothing, 0)
        status = swModelDocExt.SelectByID2("", "VERTEX", -0.085, -0.09, 0.065, True, 8, Nothing, 0)

        'Insert move/copy body feature
        swFeature = swFeatureManager.InsertMoveCopyBody2(0, 0, 0, 0, -0.085, 0, 0.065, 0, 0, 0, True, 1)
        swFeature = swPart.FeatureByName("Body-Move/Copy1")

        'Roll forward the feature and get and set move/copy body feature data
        swMoveCopyBodyFeatureData = swFeature.GetDefinition
        status = swMoveCopyBodyFeatureData.AccessSelections(swModel, Nothing)

        'Get the body to set
        status = swModelDocExt.SelectByID2("Extrude-Thin1", "SOLIDBODY", 0, 0, 0, False, 0, Nothing, swSelectOption_e.swSelectOptionDefault)
        bodyArr(0) = swSelectionMgr.GetSelectedObject6(1, -1)
        swModel.ClearSelection2(True)
        swMoveCopyBodyFeatureData.Bodies = bodyArr(0)
        If IsNothing(bodyArr) Then
            Debug.Print("Body not set.")
        Else
            Debug.Print("Body set.")
            aBody = bodyArr(0)
            Debug.Print("Name of body set: " & aBody.Name)
        End If

        'Roll back the feature
        swMoveCopyBodyFeatureData.ReleaseSelectionAccess()

        swModel.ViewZoomtofit2()

    End Sub

    ''' <summary>
    ''' The SldWorks swApp variable is pre-assigned for you.
    ''' </summary>
    Public swApp As SldWorks

End Class

```